

# **LAPORAN MILESTONE 2 TUGAS BESAR IF2224 TEORI BAHASA FORMAL DAN OTOMATA**

## **ARION COMPILER**

### **Syntax Analysis**

DEADLINE: Kamis, 7 Mei 2026 Pukul 23:59 WIB



Dipersiapkan oleh:

Kelompok AZZAM | Kode MAR | Kelas 01

|          |                       |
|----------|-----------------------|
| 13524017 | Aziza Dharma Putri    |
| 13424053 | Ahmad Zaky Robbani    |
| 13524063 | Marcel Luther Sitorus |
| 13524081 | Alya Nur Rahmah       |

**PROGRAM STUDI TEKNIK INFORMATIKA  
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA  
INSTITUT TEKNOLOGI BANDUNG  
JL. GANESA 10, BANDUNG 40132  
2026**

## DAFTAR ISI

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>DAFTAR ISI.....</b>                                     | <b>2</b>   |
| <b>TEORI SINGKAT.....</b>                                  | <b>3</b>   |
| 1.1. Context-Free Grammar (CFG).....                       | 3          |
| 1.2. Parse Tree.....                                       | 4          |
| 1.3. Ambiguitas pada Grammar dan Transformasi Grammar..... | 4          |
| 1.4. Top-Down Parsing.....                                 | 5          |
| 1.5. Recursive Descent Parsing.....                        | 6          |
| <b>PERANCANGAN DAN IMPLEMENTASI.....</b>                   | <b>7</b>   |
| 2.1. Perancangan.....                                      | 7          |
| 2.2. Perancangan Aturan Produksi.....                      | 7          |
| 2.3. Perancangan Analisis Sintaks (Parser).....            | 11         |
| 2.4. Implementasi.....                                     | 12         |
| 2.4.1. Alur Program.....                                   | 12         |
| 2.4.2. Kode Program Utama.....                             | 13         |
| 2.4.3. Kode Kelas Parse Tree.....                          | 16         |
| 2.4.4. Kode Kelas Parser.....                              | 22         |
| <b>PENGUJIAN.....</b>                                      | <b>33</b>  |
| 3.1. Pengujian Pertama.....                                | 33         |
| 3.2. Pengujian Kedua.....                                  | 34         |
| 3.3. Pengujian Ketiga.....                                 | 37         |
| 3.4. Pengujian Keempat.....                                | 43         |
| 3.5. Pengujian Kelima.....                                 | 47         |
| 3.6. Pengujian Keenam.....                                 | 53         |
| 3.7. Pengujian Ketujuh.....                                | 61         |
| 3.8. Pengujian Kedelapan.....                              | 68         |
| 3.9. Pengujian Kesembilan.....                             | 72         |
| 3.10. Pengujian Kesepuluh.....                             | 82         |
| 3.11. Pengujian Kesebelas.....                             | 99         |
| 3.12. ....                                                 | 100        |
| <b>KESIMPULAN DAN SARAN.....</b>                           | <b>101</b> |
| 4.1. Kesimpulan.....                                       | 101        |
| 4.2. Saran.....                                            | 101        |
| <b>LAMPIRAN.....</b>                                       | <b>102</b> |
| <b>DAFTAR PUSTAKA.....</b>                                 | <b>103</b> |

# BAB I

## TEORI SINGKAT

### 1.1. Context-Free Grammar (CFG)

Secara formal *Context-Free Grammar* (CFG) dapat direpresentasikan ke dalam *quadruple* sebagai berikut:

$$G = (V, T, P, S)$$

dengan komponen-komponen sebagai berikut:

1. **V** adalah simbol untuk variabel. Dalam *grammar*, variabel merupakan semacam *placeholder* pada string yang dihasilkan oleh *grammar*. Sesuai dengan namanya variabel masih dapat disubstitusikan atau diturunkan menjadi suatu simbol yang merupakan simbol aktual yang valid pada *string* yang dihasilkan oleh *grammar*. Biasanya variabel pada *grammar* disimbolkan dengan huruf kapital (S, A, ..., Z).
2. **T** adalah simbol untuk terminal. Terminal ini merupakan simbol aktual yang sudah tidak dapat diturunkan ataupun disubstitusikan. Terminal inilah yang menjadi *string* aktual yang dihasilkan oleh *grammar*. Karena variabel merupakan simbol yang masih dapat disubstitusikan, maka variabel ini dapat dinyatakan sebagai non-terminal. Terminal ini bisa mencakup simbol huruf, angka, dan token atau pun ekspresi.
3. **P** merupakan sekumpulan aturan produksi dari *grammar*. Pada bagian inilah yang mengatur bagaimana variabel diturunkan. Variabel dapat diturunkan ke beberapa bentuk string maupun terminal dan masing-masing penurunan tersebut dapat diterapkan secara bebas dalam menghasilkan *string* dari *grammar*. Inilah mengapa aturan produksi ini dinyatakan *context-free*. Aturan tersebut dinyatakan sebagai berikut

$$A \rightarrow \alpha, \alpha \in (V \cup T)^*$$

Alpha ( $\alpha$ ) merupakan elemen dari variabel dan/atau terminal yang dapat diproses secara rekursi sebanyak 0 sampai n, n adalah bilangan bulat berhingga. Perlu diperhatikan bahwa sisi kiri dari aturan produksi adalah satu variabel saja.

4. **S** adalah simbol mulai dari *grammar*. Simbol mulai ini menggunakan variabel. Simbol mulai ini menyatakan awal mula penurunan variabel untuk menghasilkan string yang sesuai dengan aturan produksi.

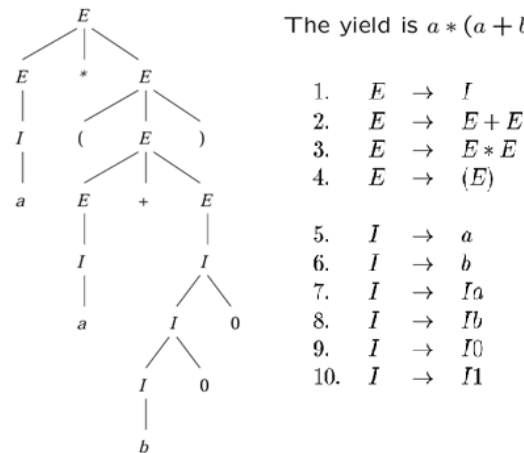
Bahasa yang dihasilkan dari CFG dapat dinotasikan sebagai berikut:

$$L(G) = \{w \Rightarrow T^* : S \Rightarrow_G^* w\}$$

Notasi tersebut menyatakan bahwa *string* w merupakan bahasa dari *grammar* jika w dapat dihasilkan dari penurunan variabel S; S adalah simbol mulai dari *grammar*.

## 1.2. Parse Tree

Parse Tree merupakan pohon yang merepresentasikan penurunan dari aturan produksi yang ada pada CFG hingga menghasilkan komponen-komponen terminal. Seluruh bahasa  $L(G)$  yang dihasilkan oleh grammar dapat direpresentasikan oleh *parse tree*.



Gambar 1.2 Contoh Parse Tree

*Yield* pada gambar tersebut dapat dinyatakan sebagai string dari bahasa  $L(G)$  yang dihasilkan oleh *grammar*. Terdapat dua pendekatan untuk menghasilkan *parse tree*, yaitu *leftmost derivation* dan *rightmost derivation*. *Leftmost derivation* merupakan tipe penurunan variabel yang mengutamakan penurunan variabel paling kiri dahulu lalu ke variabel paling kanan, sementara *rightmost derivation* sebaliknya, prioritas penurunan variabel diutamakan dari kanan ke kiri. Idealnya setiap *string* yang termasuk pada bahasa  $L(G)$  menghasilkan *parse tree* walaupun menggunakan *leftmost derivation* maupun *rightmost derivation*.

## 1.3. Ambiguitas pada Grammar dan Transformasi Grammar

Apabila suatu string grammar memiliki lebih dari satu parse tree pada saat metode penurunan *leftmost derivation* dan *rightmost derivation* diterapkan, maka dapat dinyatakan bahwa *grammar* tersebut ambigu. Ambiguitas tersebut tidak selalu dapat dihilangkan dan belum terdapat algoritma seragam untuk menghilangkan ambiguitas tersebut. Salah satu cara untuk menghilangkan ambiguitas tersebut adalah dengan melakukan transformasi aturan grammar. Contoh penerapan transformasi grammar ini dapat dilihat untuk aturan aritmatika terkait tingkat prioritas perkalian dan penjumlahan pada suatu string yang merupakan bagian dari bahasa yang dihasilkan oleh grammar.

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E * E \\ E &\rightarrow (E) \\ E &\rightarrow id \end{aligned}$$

Menggunakan aturan produksi tersebut, *string* berikut

$$id + id * id$$

dapat diinterpretasikan ke dalam dua cara yang berbeda

$$(id + id) * id$$
$$id + (id * id)$$

Untuk menyelesaikan ambiguitas tersebut, perlu mentransformasi *grammar* dengan menambahkan prioritas dalam simbol aritmatika sebagai berikut

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

Dalam aturan produksi tersebut, E menangani operasi penjumlahan dan T menangani aturan perkalian. Struktur tersebut memastikan bahwa perkalian memiliki prioritas yang lebih tinggi. Oleh karena itu *string* berikut

$$id + id * id$$

Akan di-*parsing* menjadi

$$id + (id * id)$$

Transformasi *grammar* ditujukan bukan untuk mengubah bahasa yang dihasilkan oleh *grammar* tetapi untuk mempermudah penggunaan pada saat melakukan *parsing*, mengurangi ambiguitas *grammar*, dan menyesuaikan dengan algoritma *parsing* yang digunakan.

#### 1.4. Top-Down Parsing

*Top-Down Parsing* merupakan kategori algoritma parsing yang memvalidasi apakah suatu *string* merupakan *string* yang dapat dihasilkan oleh aturan produksi dengan melakukan penurunan dari simbol mulai (*S*) *grammar*. Beberapa algoritma *parsing* yang menggunakan pendekatan ini adalah *recursive descent parsing* dan *LL(1) parsing*. Secara umum *recursive descent parsing* menerapkan metode *backtracking*, sementara *LL(1) parsing* menggunakan tabel *parsing* sehingga pencarian untuk menghasilkan urutan penggunaan aturan produksi yang benar langsung berdasarkan tabel *parsing* dan tidak melakukan *backtracking*.

## 1.5. Recursive Descent Parsing

*Recursive Descent parsing* adalah metode penguraian top-down di mana setiap nonterminal dalam tata bahasa bebas konteks direpresentasikan oleh prosedur penguraian. Pengurai dimulai dari simbol awal dan secara rekursif memperluas nonterminal dengan menerapkan aturan produksi hingga token masukan cocok. Metode ini umum digunakan dalam analisis sintaksis karena implementasinya sederhana dan mengikuti struktur tata bahasa dengan cermat. Namun, penguraian penurunan rekursif memiliki keterbatasan, terutama ketika menangani tata bahasa rekursif kiri, karena tata bahasa tersebut dapat menyebabkan panggilan rekursif tak terbatas. Oleh karena itu, transformasi tata bahasa seperti menghilangkan rekursif kiri dan menerapkan faktorisasi kiri seringkali diperlukan untuk membuat tata bahasa tersebut sesuai untuk penguraian penurunan rekursif.

Terdapat pola umum yang diterapkan pada *recursive Descent parsing*. Sebagai contoh, apabila diketahui aturan produksi sebagai berikut.

$$E \rightarrow T \mid T + E$$
$$T \rightarrow \text{int} \mid \text{int} * T \mid ( E )$$

Maka *pseudocode* dari *recursive Descent parsing* adalah sebagai berikut.

```
parseE():
  coba parseT()
  atau coba parseT() diikuti '+', dan parseE()

parseT():
  coba cocokkan dengan 'int'
  atau coba cocokkan dengan 'int' diikuti '*', dan parseT()
  atau coba cocokkan dengan '(', diikuti parseE(), dan ')'
```

Pola tersebut menyebabkan aturan produksi seperti

$$S \rightarrow S\alpha$$

Menghasilkan *pseudocode* berikut

```
parseS():
  coba parseS()
  atau coba '\alpha'
```

*Pseudocode* tersebut akan mengalami pengulangan tak hingga karena tidak ada kondisi berhenti yang jelas untuk “parseS()” dan baris “atau coba '\alpha'” tidak pernah tercapai. Permasalahan tersebut dapat diatasi dengan transformasi aturan produksi sebagai berikut

$$S \rightarrow S'$$
$$S' \rightarrow \alpha$$

## BAB II

### PERANCANGAN DAN IMPLEMENTASI

#### 2.1. Perancangan

Pada tahap perancangan ini *lexer* telah diimplementasikan, sehingga yang perlu dirancang adalah *parser* untuk bahasa Arion. Parser dirancang supaya bisa diimplementasikan menggunakan bahasa C++. Dan metode parsing yang digunakan adalah *Recursive Descent Parsing*, bentuk algoritma *Top-Down Parsing* yang memanfaatkan sifat rekursif.

Untuk mendukung pengimplementasian dari parser secara lengkap, maka *parse tree* dan juga aturan produksi perlu dirancang terlebih dahulu. Secara umum *parse tree* dirancang memiliki informasi berikut.

1. *Node kind*, jenis node berupa terminal, non-terminal, dan error
2. Label dari node, label ini berupa token yang dihasilkan oleh *lexer*.
3. *Line*, baris tempat token berada pada *source code* yang ditinjau oleh parser.
4. *Column*, kolom tempat token berada pada *source code*.

Sementara aturan produksi akan menentukan alur pemilihan aturan produksi yang sesuai dengan *source code* yang sedang ditinjau, sehingga dapat ditinjau apakah token berada pada urutan yang benar, dengan kata lain benar secara sintaks. Aturan produksi ini mengikuti notasi yang berlaku pada *Context-Free Grammar*(CFG).

#### 2.2. Perancangan Aturan Produksi

Perancangan aturan produksi ini dibuat untuk merepresentasikan alur yang diimplementasikan pada file `parser.cpp`. Terdapat beberapa transformasi yang diterapkan pada aturan produksi, terutama aturan produksi yang mengandung variabel sisi kiri pada sisi kanannya untuk menghindari *infinite loop*. Berikut adalah perancangan aturan produksi final yang diterapkan pada `parser.cpp`.

```
Program
-> ProgramHeader Block PERIOD

ProgramHeader
-> PROGRAMSY IDENT SEMICOLON
   | PROGRAMSY IDENT LPARENT IdentifierList RPARENT SEMICOLON

IdentifierList
-> IDENT
   | IDENT COMMA IdentifierList

Block
-> DeclarationPart CompoundStatement
```

```

DeclarationPart
    -> { ConstDeclaration | TypeDeclaration | VarDeclaration |
SubprogramDeclarationPart }

ConstDeclaration
    -> CONSTSY ConstDefinition SEMICOLON { ConstDefinition SEMICOLON }

ConstDefinition
    -> IDENT BECOMES Constant

TypeDeclaration
    -> TYPESY TypeDefinition SEMICOLON { TypeDefinition SEMICOLON }

TypeDefinition
    -> IDENT COLON Type

VarDeclaration
    -> VARSY VariableDeclaration SEMICOLON { VariableDeclaration SEMICOLON }

VariableDeclaration
    -> IdentifierList COLON Type

Type
    -> IDENT
    | ArrayType
    | RecordType
    | Enumerated

ArrayType
    -> ARRAYSY LBRACK Range { COMMA Range } RBRACK OFSY Type

Range
    -> Constant PERIOD PERIOD Constant

Enumerated
    -> LPARENT IdentifierList RPARENT

RecordType
    -> RECORDSY FieldList ENDSY

FieldList
    -> VariableDeclaration { SEMICOLON VariableDeclaration } [ SEMICOLON ]

SubprogramDeclarationPart
    -> SubprogramDeclaration SEMICOLON { SubprogramDeclaration SEMICOLON }

SubprogramDeclaration
    -> ProcedureDeclaration
    | FunctionDeclaration

ProcedureDeclaration
    -> ProcedureHeading SEMICOLON Block

FunctionDeclaration
    -> FunctionHeading SEMICOLON Block

```



```

ProcedureHeading
-> PROCEDURESY IDENT
   | PROCEDURESY IDENT FormalParameterList

FunctionHeading
-> FUNCTIONSY IDENT COLON IDENT
   | FUNCTIONSY IDENT FormalParameterList COLON IDENT

FormalParameterList
-> LPARENT ParameterGroup { SEMICOLON ParameterGroup } RPARENT

ParameterGroup
-> IdentifierList COLON IDENT

CompoundStatement
-> BEGINSY StatementList ENDSY

StatementList
-> Statement { SEMICOLON Statement }

Statement
-> AssignmentStatement
   | ProcedureFunctionCall
   | CompoundStatement
   | IfStatement
   | CaseStatement
   | RepeatStatement
   | WhileStatement
   | ForStatement
   | Empty

Empty
-> ε

AssignmentStatement
-> Variable BECOMES Expression

ProcedureFunctionCall
-> IDENT
   | IDENT LPARENT [ ParameterList ] RPARENT
   | Variable

IfStatement
-> IFSY Expression THENSY Statement
   | IFSY Expression THENSY Statement ELSESY Statement

CaseStatement
-> CASESY Expression OFSY CaseBlock { SEMICOLON CaseBlock } [ SEMICOLON ]
ENDSY

CaseBlock
-> Constant COLON Statement

RepeatStatement
-> REPEATSY StatementList UNTILSY Expression

```

```

WhileStatement
  -> WHILESY Expression DOSY Statement

ForStatement
  -> FORSY IDENT BECOMES Expression TOSY Expression DOSY Statement
  | FORSY IDENT BECOMES Expression DOWNTOSY Expression DOSY Statement

ParameterList
  -> Expression { COMMA Expression }

Expression
  -> SimpleExpression
  | SimpleExpression RelationalOperator SimpleExpression

RelationalOperator
  -> EQL
  | NEQ
  | LSS
  | LEQ
  | GTR
  | GEQ

SimpleExpression
  -> Term { AdditiveOperator Term }

AdditiveOperator
  -> PLUS
  | MINUS
  | ORSY

Term
  -> Factor { MultiplicativeOperator Factor }

MultiplicativeOperator
  -> TIMES
  | IDIV
  | RDIV
  | IMOD
  | ANDSY

Factor
  -> LPARENT Expression RPARENT
  | NOTSY Factor
  | Variable
  | ProcedureFunctionCall
  | Constant

Variable
  -> IDENT { Selector }

Selector
  -> LBRACK IndexList RBRACK
  | PERIOD IDENT

IndexList
  -> Expression { COMMA Expression }

```

```
Constant  
-> [ PLUS | MINUS ] ConstantValue
```

```
ConstantValue  
-> IDENT  
| INTCON  
| REALCON  
| CHARCON  
| STRING
```

Keterangan:

1. { X } artinya X dapat berulang sebanyak nol sampai beberapa kali.
2. [ X ] artinya X adalah opsional.
3.  $\epsilon$  artinya aturan produksi untuk kasus token kosong.
4. *Starting Variable* dari aturan produksi tersebut adalah variabel Program

### 2.3. Perancangan Analisis Sintaks (Parser)

Analisis sintaks atau *parser* yang diimplementasikan pada tugas ini adalah *parser* yang mampu menghasilkan *parse tree*. Sesuai dengan spesifikasi pada Milestone 2, *parser* dibantu oleh *lexer* yang telah diimplementasikan pada Milestone 1 dan menghasilkan *parse tree* yang dapat disimpan ke dalam file .txt.

Secara umum, alur kerja parser adalah sebagai berikut:

1. Program membaca file source code karakter demi karakter dengan bantuan *lexer*.
2. Setiap karakter menjadi input bagi DFA untuk menentukan state berikutnya.
3. Selama masih terdapat transisi yang valid, karakter akan digabungkan menjadi sebuah lexeme.
4. Ketika tidak ada transisi lanjutan, maka lexeme akan diperiksa pada dua kondisi, jika berada pada state final, maka lexeme akan dikonversi menjadi token. Kemudian, jika tidak, maka dianggap sebagai token unknown.
5. Setelah kumpulan token didapatkan dari *source code*, akan dilakukan *top-down parsing* oleh *parser*. Untuk menentukan apakah urutan token tersebut sesuai dengan aturan produksi yang telah ditetapkan.
6. Jika ditemukan kesalahan dalam proses *top-down parsing*, dengan kata lain aturan produksi tidak dapat membentuk token sesuai dengan urutan *source code*, maka *LexerException* akan dilempar.

Pendekatan tersebut menjelaskan keterhubungan antara *lexer* dengan *parser*, bagaimana kedua sistem tersebut dapat melakukan pengecekan kode hingga level sintaks.

## 2.4. Implementasi

### 2.4.1. Alur Program

Program dimulai dengan membaca perintah command line yang diberikan pengguna. Perintah utama berupa file source code Arion berekstensi .txt, sedangkan perintah tambahan digunakan untuk menentukan file output token (-o), file output parse tree (-p), serta mode --lex-only apabila hanya ingin menjalankan lexical analyzer tanpa parser. Jika argumen yang diberikan tidak valid atau file input tidak ditemukan, maka program akan menampilkan pesan error dan menghentikan eksekusi.

Setelah validasi argumen selesai, program menentukan lokasi file output. Jika pengguna tidak memberikan nama file output secara eksplisit, maka program akan membuat path output otomatis berdasarkan nama file input. Output token disimpan pada folder milestone 1, sedangkan output parse tree disimpan pada folder milestone 2. Program juga memastikan direktori tujuan sudah tersedia menggunakan `std::filesystem`.

Tahap berikutnya adalah proses lexical analysis. Program membuka file input kemudian menginisialisasi objek DFA yang akan digunakan oleh lexer. Konfigurasi DFA dimuat dari file `config/config_lexer.txt`, sehingga seluruh state, final state, dan aturan transisi dapat diatur secara fleksibel tanpa mengubah kode program utama. Setelah itu, objek Lexer dibuat dengan memanfaatkan file input, DFA, dan file output token.

Lexer membaca source code karakter demi karakter. Pada setiap karakter yang dibaca, DFA menentukan state berikutnya berdasarkan tabel transisi yang telah dimuat sebelumnya. Selama masih terdapat transisi yang valid, karakter akan terus digabungkan menjadi sebuah lexeme sesuai prinsip maximal munch (longest match). Ketika state akhir tercapai, lexeme akan dikonversi menjadi token yang sesuai dan disimpan ke dalam hasil lexer serta file output token.

Setelah seluruh karakter selesai diproses, lexer menghasilkan kumpulan token yang merepresentasikan source code. Program kemudian melakukan pengecekan terhadap token-token tersebut sebelum masuk ke parser. Token komentar (COMMENT) akan diabaikan karena tidak mempengaruhi analisis sintaks. Selain itu, jika ditemukan token UNKNOWN, maka program akan menghasilkan lexical error dan proses parsing tidak dilanjutkan. Kemudian program melanjutkan ke tahap syntax analysis menggunakan *top down parsing*. Seluruh token valid dimasukkan ke dalam objek Parser. Parser melakukan analisis top-down berdasarkan grammar yang telah diimplementasikan pada fungsi-fungsi parser. Setiap non-terminal direpresentasikan oleh fungsi parsing tersendiri yang akan memeriksa urutan token sesuai aturan produksi.

Selama proses parsing berlangsung, parser membangun struktur parse tree yang merepresentasikan hierarki sintaks program Arion. Node-node parse tree terdiri atas terminal dan non-terminal sesuai grammar yang digunakan. Jika urutan token tidak sesuai dengan aturan produksi, parser akan melempar `ParseException` dengan pesan error yang informatif. Apabila parsing berhasil dilakukan, parse tree akan dicetak ke terminal dan juga disimpan ke file output parse tree.

### 2.4.2. Kode Program Utama

#### Implementasi

```
#include <filesystem>
#include <fstream>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

#include "lexer/dfa.hpp"
#include "lexer/lexer.hpp"
#include "lexer/lexer_exception.hpp"
#include "parser/parser.hpp"
#include "parser/parse_tree.hpp"

namespace fs = std::filesystem;

void printUsage(const char* programName) {
    std::cerr
        << "Usage:\n"
        << " " << programName
        << " <program.txt> [-o <token_output.txt>] [-p <parse_tree_output.txt>]
[--lex-only]\n\n"
        << "Options:\n"
        << "  -o <file>      Save lexer/token output to <file>\n"
        << "  -p <file>      Save parser/parse-tree output to <file>\n"
        << "  --lex-only     Run only lexer, do not run parser\n";
}

std::string defaultOutputPath(const std::string& inputFilename,
                             const std::string& milestoneFolder) {
    fs::path inputPath(inputFilename);
    std::string stem = inputPath.stem().string();

    if (stem.rfind("input", 0) == 0) {
        stem = "output" + stem.substr(5);
    }

    return (fs::path("test") / milestoneFolder / "output" / (stem +
".txt")).string();
}

void ensureParentDirectoryExists(const std::string& filename) {
    fs::path path(filename);

    if (path.has_parent_path()) {
        fs::create_directories(path.parent_path());
    }
}

bool isIgnoredByParser(const Token& token) {
    return token.get_type_name() == "COMMENT";
}

bool isLexicalErrorToken(const Token& token) {
    return token.get_type_name() == "UNKNOWN";
}

int main(int argc, char* argv[]) {
```

```

if (argc < 2) {
    printUsage(argv[0]);
    return 1;
}

std::string inputFilename = argv[1];
std::string tokenOutputFilename;
std::string parseTreeOutputFilename;
bool lexOnly = false;

for (int i = 2; i < argc; ++i) {
    std::string arg = argv[i];

    if (arg == "-o") {
        if (i + 1 >= argc) {
            std::cerr << "ERROR: -o membutuhkan nama file output.\n";
            printUsage(argv[0]);
            return 1;
        }

        tokenOutputFilename = argv[++i];
    } else if (arg == "-p") {
        if (i + 1 >= argc) {
            std::cerr << "ERROR: -p membutuhkan nama file output parse tree.\n";
            printUsage(argv[0]);
            return 1;
        }

        parseTreeOutputFilename = argv[++i];
    } else if (arg == "--lex-only") {
        lexOnly = true;
    } else {
        std::cerr << "ERROR: Argumen tidak dikenal: " << arg << "\n";
        printUsage(argv[0]);
        return 1;
    }
}

if (tokenOutputFilename.empty()) {
    tokenOutputFilename = defaultOutputPath(inputFilename, "milestone1");
}

if (parseTreeOutputFilename.empty()) {
    parseTreeOutputFilename = defaultOutputPath(inputFilename, "milestone2");
}

std::ifstream inputFile(inputFilename);
if (!inputFile.is_open()) {
    std::cerr << "Gagal membuka file input: " << inputFilename << "\n";
    return 1;
}

try {
    ensureParentDirectoryExists(tokenOutputFilename);

    std::ofstream tokenOutputFile(tokenOutputFilename);
    if (!tokenOutputFile.is_open()) {
        std::cerr << "Gagal membuka file output token: " << tokenOutputFilename
        << "\n";
        return 1;
    }
}

```

```

auto dfa = std::make_shared<DFA>();
dfa->loadConfig("config/config_lexer.txt");

Lexer lexer(inputFile, dfa, &tokenOutputFile);

while (!lexer.eof()) {
    lexer.process_next_token();
}

tokenOutputFile.close();

std::cout << "Lexing selesai.\n";
std::cout << "Output token disimpan di: " << tokenOutputFilename << "\n";

if (lexOnly) {
    return 0;
}

std::vector<Token> parserTokens;
parserTokens.reserve(lexer.getResult().size());

for (const Token& token : lexer.getResult()) {
    if (isIgnoredByParser(token)) {
        continue;
    }

    if (isLexicalErrorToken(token)) {
        std::cerr << "Lexical error: token tidak dikenal "
                    << token.to_string() << "\n";
        return 1;
    }

    parserTokens.push_back(token);
}

Parser parser(parserTokens);
parse_tree::NodePtr root = parser.parse();

ensureParentDirectoryExists(parseTreeOutputFilename);

std::ofstream parseTreeOutputFile(parseTreeOutputFilename);
if (!parseTreeOutputFile.is_open()) {
    std::cerr << "Gagal membuka file output parse tree: "
                << parseTreeOutputFilename << "\n";
    return 1;
}

parse_tree::printTree(root.get(), std::cout);
parse_tree::printTree(root.get(), parseTreeOutputFile);
parseTreeOutputFile.close();

std::cout << "Parsing selesai.\n";
std::cout << "Output parse tree disimpan di: "
            << parseTreeOutputFilename << "\n";
} catch (const LexerException& e) {
    std::cerr << e.full_message() << "\n";
    return 1;
} catch (const ParseException& e) {
    std::cerr << e.full_message() << "\n";
    return 1;
} catch (const std::exception& e) {
    std::cerr << "ERROR: " << e.what() << "\n";
}

```

```

        return 1;
    }

    return 0;
}

```

### Penjelasan

Program utama berfungsi sebagai driver yang menghubungkan seluruh komponen compiler, yaitu pembacaan file input, lexical analyzer, dan parser. Program menerima source code Arion melalui argumen command line, kemudian menyediakan beberapa opsi tambahan seperti menentukan file output token, file output parse tree, dan mode khusus lexical analysis saja (--lex-only).

Pada tahap awal, program melakukan validasi argumen untuk memastikan file input tersedia. Setelah itu, program menentukan lokasi file output secara otomatis apabila pengguna tidak memberikan nama file output secara eksplisit. Direktori output juga dibuat otomatis menggunakan std::filesystem apabila belum tersedia. Selanjutnya, program membuka file input dan menginisialisasi objek DFA. DFA memuat konfigurasi state dan transisi dari file eksternal config/config\_lexer.txt. DFA ini kemudian digunakan oleh objek Lexer untuk melakukan tokenisasi source code. Proses lexical analysis dilakukan secara iteratif menggunakan process\_next\_token() hingga seluruh isi file selesai diproses.

Setelah lexical analysis selesai, seluruh token hasil lexer diperiksa kembali sebelum masuk ke parser. Token komentar akan diabaikan karena tidak diperlukan dalam analisis sintaks, sedangkan token bertipe UNKNOWN dianggap sebagai lexical error sehingga program langsung menghentikan eksekusi. Tahap berikutnya adalah syntax analysis menggunakan *top down parsing*. Parser menerima vector token hasil lexer dan memprosesnya berdasarkan aturan produksi grammar Arion. Jika parsing berhasil dilakukan, parser menghasilkan parse tree yang kemudian dicetak ke terminal serta disimpan ke file output.

Program juga dilengkapi dengan exception handling menggunakan try-catch untuk menangani berbagai jenis error, seperti LexerException, ParseException, maupun exception umum lainnya. Dengan demikian, program dapat memberikan pesan error yang informatif apabila terjadi kesalahan selama proses lexical analysis maupun syntax analysis.

### 2.4.3. Kode Kelas Parse Tree

#### Header

```

// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github
#ifndef PARSE_TREE_HPP
#define PARSE_TREE_HPP

#include "../lexer/token.hpp"

#include <iosfwd>
#include <memory>
#include <string>
#include <vector>

namespace parse_tree {
enum class NodeKind {

```



```

    Terminal,
    NonTerminal,
    Error
};

enum class NonTerminal {
    // Berisi jenis node nonterminal
};

class Node {
public:
    Node(NodeKind kind, std::string label, std::string value = "", int line = -1,
        int column = -1);

    // Takes ownership of raw child pointers produced by
    createTerminal/createNonTerminal.
    Node* addChild(Node* child);
    Node* addChild(std::unique_ptr<Node> child);

    NodeKind getKind() const;
    bool isTerminal() const;
    bool isNonTerminal() const;
    bool isError() const;
    const std::string& getLabel() const;
    const std::string& getValue() const;
    int getLine() const;
    int getColumn() const;
    const std::vector<std::unique_ptr<Node>>& getChildren() const;

    std::string toString() const;
    void print(std::ostream& out, int depth = 0) const;

private:
    NodeKind kind;
    std::string label;
    std::string value;
    int line;
    int column;
    std::vector<std::unique_ptr<Node>> children;
};

using NodePtr = std::unique_ptr<Node>;

const char* toString(NonTerminal symbol);

NodePtr makeNonTerminal(NonTerminal symbol);
NodePtr makeNonTerminal(const std::string& label);
NodePtr makeTerminal(const std::string& tokenType, const std::string& lexeme = "",
    int line = -1, int column = -1);
NodePtr makeTerminal(const Token& token);
NodePtr makeError(const std::string& message, int line = -1, int column = -1);

Node* createNonTerminal(NonTerminal symbol);
Node* createNonTerminal(const std::string& label);
Node* createTerminal(const std::string& tokenType, const std::string& lexeme = "",
    int line = -1, int column = -1);
Node* createTerminal(const Token& token);
Node* createError(const std::string& message, int line = -1, int column = -1);

void printTree(const Node* root, std::ostream& out);

} // namespace parse_tree

```

```
#endif
```

## Implementasi

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

using NodePtr = std::unique_ptr<Node>;

Node::Node(NodeKind kind, std::string label, std::string value, int line, int
column) : kind(kind), label(std::move(label)), value(std::move(value)),
line(line), column(column) {}
// Constructor parse tree node

Node* Node::addChild(Node* child) {
    // Menambahkan child node
    if (child == nullptr) {
        return nullptr;
    }

    children.emplace_back(child);
    return children.back().get();
}

std::string Node::toString() const {
    // Format non-terminal
    if (isNonTerminal()) {
        return "<" + label + ">";
    }

    // Format error node
    if (isError()) {
        return "<error: " + value + ">";
    }

    // Format terminal dengan lexeme
    if (!value.empty()) {
        return label + " (" + value + ")";
    }

    return label;
}

void Node::print(std::ostream& out,
                int depth) const {

    // Memberikan indentasi sesuai depth
    for (int i = 0; i < depth; ++i) {
        out << "  ";
    }

    out << toString();

    // Menampilkan posisi token
```

```

        if (line >= 0 && column >= 0) {
            out << " [" << line
                << ":" << column << "]\n";
        }

        out << '\n';

        // Print child secara rekursif
        for (const auto& child : children) {
            child->print(out, depth + 1);
        }
    }

    const char* toString(NonTerminal symbol) {
        // Mengubah enum menjadi string grammar
        switch (symbol) {

            case NonTerminal::Program:
                return "program";

            case NonTerminal::Statement:
                return "statement";

            case NonTerminal::Expression:
                return "expression";

            // ... non-terminal lainnya ...

            default:
                throw std::logic_error(
                    "Unhandled non-terminal symbol"
                );
        }
    }

    NodePtr makeNonTerminal(
        NonTerminal symbol) {

        // Membuat node non-terminal
        return std::make_unique<Node>(
            NodeKind::NonTerminal,
            toString(symbol)
        );
    }

    NodePtr makeTerminal(
        const Token& token) {

        // Membuat node terminal dari token lexer
        std::string label =
            token.get_type_name();

        std::transform(
            label.begin(),
            label.end(),
            label.begin(),

```

```

        [](unsigned char c) {
            return std::tolower(c);
        }
    );

    return makeTerminal(
        label,
        token.get_value()
    );
}

NodePtr makeError(
    const std::string& message,
    int line,
    int column) {

    // Membuat node error parser
    return std::make_unique<Node>(
        NodeKind::Error,
        "error",
        message,
        line,
        column
    );
}

Node* createNonTerminal(
    NonTerminal symbol) {

    // Membuat raw pointer non-terminal
    return makeNonTerminal(
        symbol
    ).release();
}

Node* createTerminal(
    const Token& token) {

    // Membuat raw pointer terminal
    return makeTerminal(
        token
    ).release();
}

Node* createError(
    const std::string& message,
    int line,
    int column) {

    // Membuat raw pointer error node
    return makeError(
        message,
        line,
        column
    ).release();
}

```

```

void printTree(const Node* root,
               std::ostream& out) {

    // Print seluruh parse tree
    if (root == nullptr) {
        out << "<empty tree>\n";
        return;
    }

    root->print(out);
}

/*
Fungsi lain yang digunakan:

- bool isTerminal()
  Mengecek apakah node terminal

- bool isNonTerminal()
  Mengecek apakah node non-terminal

- bool isError()
  Mengecek apakah node error

- getLabel(), getValue()
  Mengambil informasi node

- getLine(), getColumn()
  Mengambil posisi token

- getChildren()
  Mengambil child node

- makeNonTerminal(const std::string&)
  Membuat non-terminal dari string

- makeTerminal(...)
  Membuat terminal manual
*/

```

## Penjelasan

Kelas Parse Tree digunakan untuk merepresentasikan struktur sintaks program dalam bentuk pohon hasil parsing. Setiap node dibedakan menjadi Terminal, NonTerminal, dan Error menggunakan enum NodeKind. Kelas Node menjadi struktur utama parse tree yang menyimpan label node, nilai token, dan daftar child node. Penyimpanan child menggunakan std::unique\_ptr agar pengelolaan memori dilakukan otomatis dan lebih aman. Fungsi addChild() digunakan untuk menambahkan node anak ke dalam parse tree. Fungsi toString() mengubah node menjadi format string yang mudah dibaca, sedangkan fungsi print() digunakan untuk mencetak parse tree secara rekursif dengan indentasi sesuai kedalaman node.

Selain itu, helper function seperti `makeNonTerminal()` dan `makeTerminal()` digunakan untuk mempermudah pembuatan node selama proses recursive descent parsing berlangsung. Setelah parsing selesai, seluruh struktur parse tree dapat ditampilkan menggunakan fungsi `printTree()`.

#### 2.4.4. Kode Kelas Parser

##### Header

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

#ifndef PARSER_HPP
#define PARSER_HPP

#include "../lexer/token.hpp"
#include "parse_tree.hpp"

#include <stdexcept>
#include <string>
#include <vector>

class ParseException : public std::runtime_error {
public:
    int line;
    int column;

    ParseException(const std::string& message, int line = -1, int column = -1)
        : std::runtime_error(message), line(line), column(column) {}

    std::string full_message() const {
        if (line >= 0 && column >= 0) {
            return "[ParseError " + std::to_string(line) + ":" +
                std::to_string(column) + "] " + what();
        }
        return std::string("[ParseError] ") + what();
    }
};

class Parser {
public:
    explicit Parser(const std::vector<Token>& tokens);

    parse_tree::NodePtr parse();

private:
    const std::vector<Token>& tokens;
    size_t pos;
    bool atEnd() const;
    const Token& current() const;
    const Token& lookAhead(size_t offset = 1) const;
    bool check(const std::string& typeName) const;
    bool checkAny(std::initializer_list<std::string> types) const;

    Token consume();
    Token expect(const std::string& typeName);
    parse_tree::NodePtr termNode();
    parse_tree::NodePtr expectNode(const std::string& typeName);
};
```

```

bool isConstantStart() const;
bool isStatementStart() const;
bool isTypeStart() const;
bool isDeclarationStart() const;
bool isRelationalOp() const;
bool isAdditiveOp() const;
bool isMultiplicativeOp() const;

parse_tree::NodePtr parseProgram();
parse_tree::NodePtr parseProgramHeader();
parse_tree::NodePtr parseIdentifierList();

parse_tree::NodePtr parseBlock();
parse_tree::NodePtr parseDeclarationPart();

parse_tree::NodePtr parseConstDeclaration();
parse_tree::NodePtr parseConstDefinition();

parse_tree::NodePtr parseTypeDeclaration();
parse_tree::NodePtr parseTypeDefinition();

parse_tree::NodePtr parseVarDeclaration();
parse_tree::NodePtr parseFieldPart();

parse_tree::NodePtr parseType();
parse_tree::NodePtr parseArrayType();
parse_tree::NodePtr parseRange();
parse_tree::NodePtr parseEnumerated();
parse_tree::NodePtr parseRecordType();
parse_tree::NodePtr parseFieldList();

parse_tree::NodePtr parseSubprogramDeclarationPart();
parse_tree::NodePtr parseSubprogramDeclaration();
parse_tree::NodePtr parseProcedureDeclaration();
parse_tree::NodePtr parseFunctionDeclaration();
parse_tree::NodePtr parseProcedureHeading();
parse_tree::NodePtr parseFunctionHeading();
parse_tree::NodePtr parseFormalParameterList();
parse_tree::NodePtr parseParameterGroup();

parse_tree::NodePtr parseCompoundStatement();
parse_tree::NodePtr parseStatementList();
parse_tree::NodePtr parseStatement();
parse_tree::NodePtr parseAssignmentStatement(parse_tree::NodePtr varNode);
parse_tree::NodePtr parseProcedureFunctionCall();
parse_tree::NodePtr parseIfStatement();
parse_tree::NodePtr parseCaseStatement();
parse_tree::NodePtr parseCaseBlock();
parse_tree::NodePtr parseRepeatStatement();
parse_tree::NodePtr parseWhileStatement();
parse_tree::NodePtr parseForStatement();
parse_tree::NodePtr parseParameterList();

parse_tree::NodePtr parseExpression();
parse_tree::NodePtr parseRelationalOperator();
parse_tree::NodePtr parseSimpleExpression();
parse_tree::NodePtr parseAdditiveOperator();
parse_tree::NodePtr parseTerm();
parse_tree::NodePtr parseMultiplicativeOperator();
parse_tree::NodePtr parseFactor();

parse_tree::NodePtr parseVariable();

```

```

    parse_tree::NodePtr parseSelector();
    parse_tree::NodePtr parseIndexList();
    parse_tree::NodePtr parseConstant();
};

#endif

```

## Implementasi

```

// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

class ParseException : public std::runtime_error {
public:
    int line;
    int column;

    // Constructor error parser
    ParseException(const std::string& message,
                  int line = -1,
                  int column = -1);

    // Format pesan error lengkap
    std::string full_message() const;
};

class Parser {
public:
    // Constructor parser
    explicit Parser(const std::vector<Token>& tokens);

    // Memulai proses parsing
    parse_tree::NodePtr parse();

private:
    const std::vector<Token>& tokens;
    size_t pos;

    // Mengecek akhir token
    bool atEnd() const;

    // Mengambil token saat ini
    const Token& current() const;

    // Melihat token berikutnya
    const Token& lookAhead(size_t offset = 1) const;

    // Mengecek tipe token
    bool check(const std::string& typeName) const;

    // Mengonsumsi token
    Token consume();

    // Memastikan token sesuai grammar
    Token expect(const std::string& typeName);

```



```

// Membuat node terminal
parse_tree::NodePtr termNode();

// Parsing program utama
parse_tree::NodePtr parseProgram();

// Parsing expression
parse_tree::NodePtr parseExpression();

// Parsing statement
parse_tree::NodePtr parseStatement();

// Parsing variable
parse_tree::NodePtr parseVariable();
};

Parser::Parser(const std::vector<Token>& tokens)
: tokens(tokens), pos(0) {}
// Constructor parser

bool Parser::atEnd() const {
    // Mengecek apakah token habis
    return pos >= tokens.size();
}

const Token& Parser::current() const {
    // Mengambil token saat ini
    if (atEnd()) {
        static Token eof_token(
            TokenType("EOF"), ""
        );
        return eof_token;
    }

    return tokens[pos];
}

const Token& Parser::lookAhead(
    size_t offset) const {

    // Melihat token selanjutnya
    size_t idx = pos + offset;

    if (idx >= tokens.size()) {
        static Token eof_token(
            TokenType("EOF"), ""
        );
        return eof_token;
    }

    return tokens[idx];
}

bool Parser::check(
    const std::string& typeName) const {

```

```

    // Mengecek tipe token
    if (atEnd()) {
        return typeName == "EOF";
    }

    return current().get_type_name()
        == typeName;
}

Token Parser::consume() {
    // Mengambil token dan maju
    if (atEnd()) {
        throw ParseException(
            "Sudah mencapai akhir input"
        );
    }

    return tokens[pos++];
}

Token Parser::expect(
    const std::string& typeName) {

    // Validasi token grammar
    if (!check(typeName)) {

        std::string got =
            atEnd() ? "EOF"
                : current().get_type_name();

        throw ParseException(
            "Diharapkan '" +
            typeName +
            "', mendapat '" +
            got + "'"
        );
    }

    return consume();
}

NodePtr Parser::termNode() {
    // Membuat node terminal
    Token tok = consume();
    return makeTerminal(tok);
}

NodePtr Parser::parse() {
    // Entry point parser
    return parseProgram();
}

NodePtr Parser::parseProgram() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Program

```

```

    );

    // Program -> header declaration statement
    node->addChild(
        parseProgramHeader()
    );

    node->addChild(
        parseDeclarationPart()
    );

    node->addChild(
        parseCompoundStatement()
    );

    node->addChild(
        expectNode("PERIOD")
    );

    return node;
}

```

```

NodePtr Parser::parseProgramHeader() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::ProgramHeader
        );

    // Parsing:
    // program nama_program;
    node->addChild(
        expectNode("PROGRAMSY")
    );

    node->addChild(
        expectNode("IDENT")
    );

    node->addChild(
        expectNode("SEMICOLON")
    );

    return node;
}

```

```

NodePtr Parser::parseStatement() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Statement
        );

    // Parsing statement program
    if (check("IDENT")) {

        const std::string& next =
            lookAhead(1).get_type_name();
    }
}

```

```

        // Assignment statement
        if (next == "BECOMES") {

            NodePtr varNode =
                parseVariable();

            node->addChild(
                parseAssignmentStatement(
                    std::move(varNode)
                )
            );

        } else {
            // Procedure/function call
            node->addChild(
                parseProcedureFunctionCall()
            );
        }

    } else if (check("IFS")) {

        node->addChild(
            parseIfStatement()
        );

    } else if (check("WHILE")) {

        node->addChild(
            parseWhileStatement()
        );

    } else {

        // Empty statement
        node->addChild(
            makeNonTerminal(
                NonTerminal::Empty
            )
        );

    }

    return node;
}

NodePtr Parser::parseExpression() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Expression
        );

    // Parsing ekspresi matematika/logika
    node->addChild(
        parseSimpleExpression()
    );
}

```

```

        if (isRelationalOp()) {

            node->addChild(
                parseRelationalOperator()
            );

            node->addChild(
                parseSimpleExpression()
            );
        }

        return node;
    }
}

```

```

NodePtr Parser::parseFactor() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Factor
        );

    // Parsing factor expression
    if (check("LPARENT")) {

        node->addChild(termNode());

        node->addChild(
            parseExpression()
        );

        node->addChild(
            expectNode("RPARENT")
        );
    } else if (check("IDENT")) {

        node->addChild(
            parseVariable()
        );
    } else {

        node->addChild(
            termNode()
        );
    }

    return node;
}

```

```

NodePtr Parser::parseVariable() {
    NodePtr node =
        makeNonTerminal(
            NonTerminal::Variable
        );

    // Parsing variable dan selector

```

```

        node->addChild(
            expectNode("IDENT")
        );

        while (checkAny({
            "LBRACK",
            "PERIOD"
        })) {

            node->addChild(
                parseSelector()
            );
        }

        return node;
    }

void printTree(const Node* root,
               std::ostream& out) {

    // Print parse tree hasil parser
    if (root == nullptr) {
        out << "<empty tree>\n";
        return;
    }

    root->print(out);
}

/*
Fungsi parser lain yang digunakan:

- parseIdentifierList()
  Parsing daftar identifier

- parseBlock()
  Parsing block program

- parseDeclarationPart()
  Parsing deklarasi program

- parseConstDeclaration()
  Parsing konstanta

- parseTypeDeclaration()
  Parsing tipe data

- parseVarDeclaration()
  Parsing variabel

- parseArrayType()
  Parsing array

- parseRecordType()
  Parsing record

```

- `parseCompoundStatement()`  
Parsing begin-end
- `parseIfStatement()`  
Parsing if statement
- `parseCaseStatement()`  
Parsing case statement
- `parseRepeatStatement()`  
Parsing repeat-until
- `parseWhileStatement()`  
Parsing while loop
- `parseForStatement()`  
Parsing for loop
- `parseParameterList()`  
Parsing parameter function/procedure
- `parseTerm(), parseFactor()`  
Parsing operasi matematika
- `parseSelector()`  
Parsing array index / field record
- `parseConstant()`  
Parsing konstanta
- `isRelationalOp()`  
Mengecek operator relasional
- `isAdditiveOp()`  
Mengecek operator penjumlahan
- `isMultiplicativeOp()`  
Mengecek operator perkalian

\*/

## Penjelasan

Kelas Parser digunakan untuk melakukan proses parsing terhadap token hasil lexer menggunakan metode *top down parsing*. Parser akan membaca token satu per satu berdasarkan grammar bahasa Arion yang telah ditentukan dan membangun struktur parse tree menggunakan node non-terminal maupun terminal. Class `ParseException` digunakan untuk menangani kesalahan sintaks selama parsing berlangsung. Error yang dihasilkan mencakup pesan kesalahan beserta posisi baris dan kolom token agar mempermudah debugging program.

Fungsi dasar seperti `current()`, `lookAhead()`, `consume()`, dan `expect()` berfungsi untuk mengelola proses pembacaan token. Fungsi `expect()` memiliki peran penting untuk memastikan token yang dibaca sesuai dengan grammar, sedangkan `consume()` digunakan untuk mengambil token dan memajukan posisi parser. Fungsi `parse()` merupakan entry point parser yang akan memulai parsing dari grammar paling atas yaitu `parseProgram()`. Selanjutnya parser akan memanggil fungsi parsing lain secara rekursif seperti `parseDeclarationPart()`, `parseStatement()`, `parseExpression()`, dan `parseVariable()` sesuai struktur

grammar program. Pada bagian statement, parser mampu mengenali berbagai bentuk statement seperti assignment, procedure/function call, percabangan if, perulangan while, repeat, for, serta compound statement begin-end. Pada bagian expression, parser memproses ekspresi matematika dan logika menggunakan hirarki recursive descent mulai dari Expression, SimpleExpression, Term, hingga Factor sehingga prioritas operator tetap terjaga.

Selama parsing berlangsung, setiap hasil parsing akan disimpan ke dalam parse tree menggunakan helper function seperti makeNonTerminal() dan makeTerminal(). Parse tree tersebut kemudian dapat dicetak menggunakan fungsi printTree() untuk menampilkan struktur sintaks program secara hierarkis.



## BAB III PENGUJIAN

Mangatsss Zizaaaa aku padamu gilaa makasiih udah terlahir ke dunia iniii gacor banget eaaa when yhhh sehebat Ziza hehehehehe semangaat

### 3.1. Pengujian Pertama

| Input (input1.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Output (parse_tree1.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt;  input1.txt 1  program HelloWorld; 2  var 3    a, b: integer; 4  begin 5    a := 5; 6    b := a + 10; 7    writeln('Result = ', b); 8  end. </pre>                                                                                                                                                                                                                                                                                                                          | <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  parse_tree1.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4        ident (HelloWorld) 5          semicolon (;) 6        &lt;declaration-part&gt; 7          &lt;var-declaration&gt; </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Output (token1.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  token1.txt 1  programsy 2  ident (HelloWorld) 3  semicolon 4  varsy 5  ident (a) 6  comma 7  ident (b) </pre> <p> programsy<br/> ident (HelloWorld)<br/> semicolon<br/> varsy<br/> ident (a)<br/> comma<br/> ident (b)<br/> colon<br/> ident (integer)<br/> semicolon<br/> beginsy<br/> ident (a)<br/> becomes<br/> intcon (5)<br/> semicolon<br/> ident (b)<br/> becomes<br/> ident (a)<br/> plus<br/> intcon (10)<br/> semicolon<br/> ident (writeln)<br/> lparent </p> | <pre> &lt;program&gt; &lt;program-header&gt;   programsy (program)   ident (HelloWorld)   semicolon (;) &lt;declaration-part&gt;   &lt;var-declaration&gt;     varsy (var)     &lt;identifier-list&gt;       ident (a)       comma (,)       ident (b)       colon (:)     &lt;type&gt;       ident (integer)       semicolon (;)   &lt;compound-statement&gt;     beginsy (begin)     &lt;statement-list&gt;       &lt;statement&gt;         &lt;assignment-statement&gt;           &lt;variable&gt;             ident (a)           becomes (:=)           &lt;expression&gt;             &lt;simple-expression&gt;               &lt;term&gt;                 &lt;factor&gt;                   intcon (5)             semicolon (;)           &lt;statement&gt;             &lt;assignment-statement&gt;               &lt;variable&gt;                 ident (b) </pre> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| string ('Result = ')<br>comma<br>ident (b)<br>rparent<br>semicolon<br>endsy<br>period                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a)<br><additive-operator><br>plus (+)<br><term><br><factor><br>intcon (10)<br>semicolon (;)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br>string ('Result = ')<br>comma (,)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (b)<br>rparent ())<br>semicolon (;)<br><statement><br><empty><br>endsy (end)<br>period (.) |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <p>Pengujian pertama bertujuan untuk memastikan parser dapat mengenali struktur program Arion paling dasar. Program ini memuat header program, deklarasi variabel, assignment statement, ekspresi aritmatika sederhana, dan pemanggilan prosedur writeln. Hasil token menunjukkan bahwa lexer berhasil mengklasifikasikan setiap komponen seperti programsy, ident, becomes, plus, dan string. Parse tree yang dihasilkan juga menunjukkan susunan program sesuai grammar, mulai dari &lt;program-header&gt;, &lt;declaration-part&gt;, hingga &lt;compound-statement&gt;.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

### 3.2. Pengujian Kedua

| Input (input2.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Output (parse_tree2.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt; 𐀀 input2.txt 1  program ConstAndType; 2  const 3    MAX == 100; 4    PI == 3.14; 5    INITIAL == 'A'; 6    GREETING == 'Hello'; 7  type 8    SmallRange == 1..10; 9    Season == (Spring, Summer, Fall, Winter); 10 var 11  x: integer; 12  ch: char; 13 begin 14  x := MAX; 15  ch := INITIAL; 16  writeln(x); 17 end.</pre>                                                                                                                                                                                                                                                 | <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt; 𐀀 parse_tree2.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4        ident (ConstAndType) 5          semicolon (;) 6    &lt;declaration-part&gt; 7      &lt;const-declaration&gt; 8        constsy (const) 9          &lt;const-definition&gt;</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Output (token2.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt; 𐀀 token2.txt 1  programsy 2  ident (ConstAndType) 3  semicolon 4  constsy 5  ident (MAX) 6  eql 7  intcon (100)</pre> <p>programsy<br/>ident (ConstAndType)<br/>semicolon<br/>constsy<br/>ident (MAX)<br/>eql<br/>intcon (100)<br/>semicolon<br/>ident (PI)<br/>eql<br/>realcon (3.14)<br/>semicolon<br/>ident (INITIAL)<br/>eql<br/>charcon ('A')<br/>semicolon<br/>ident (GREETING)<br/>eql<br/>string ('Hello')<br/>semicolon<br/>typesy<br/>ident (SmallRange)<br/>eql<br/>intcon (1)<br/>period<br/>period<br/>intcon (10)<br/>semicolon<br/>ident (Season)<br/>eql</p> | <pre>&lt;program&gt; &lt;program-header&gt;   programsy (program)   ident (ConstAndType)   semicolon (;) &lt;declaration-part&gt;   &lt;const-declaration&gt;     constsy (const)     &lt;const-definition&gt;       ident (MAX)       eql (==)       &lt;constant&gt;         intcon (100)       semicolon (;)     &lt;const-definition&gt;       ident (PI)       eql (==)       &lt;constant&gt;         realcon (3.14)       semicolon (;)     &lt;const-definition&gt;       ident (INITIAL)       eql (==)       &lt;constant&gt;         charcon ('A')       semicolon (;)     &lt;const-definition&gt;       ident (GREETING)       eql (==)       &lt;constant&gt;         string ('Hello')       semicolon (;)   &lt;type-declaration&gt;     typesy (type)     &lt;type-definition&gt;       ident (SmallRange)       eql (==)       &lt;type&gt;         &lt;range&gt;           &lt;constant&gt;             intcon (1)</pre> |

|                 |                        |
|-----------------|------------------------|
| lparent         | period (.)             |
| ident (Spring)  | period (.)             |
| comma           | <constant>             |
| ident (Summer)  | intcon (10)            |
| comma           | semicolon (;)          |
| ident (Fall)    | <type-definition>      |
| comma           | ident (Season)         |
| ident (Winter)  | eql (==)               |
| rparent         | <type>                 |
| semicolon       | <enumerated>           |
| varsy           | lparent ()             |
| ident (x)       | <identifier-list>      |
| colon           | ident (Spring)         |
| ident (integer) | comma (,)              |
| semicolon       | ident (Summer)         |
| ident (ch)      | comma (,)              |
| colon           | ident (Fall)           |
| ident (char)    | comma (,)              |
| semicolon       | ident (Winter)         |
| beginsy         | rparent ()             |
| ident (x)       | semicolon (;)          |
| becomes         | <var-declaration>      |
| ident (MAX)     | varsy (var)            |
| semicolon       | <identifier-list>      |
| ident (ch)      | ident (x)              |
| becomes         | colon (:)              |
| ident (INITIAL) | <type>                 |
| semicolon       | ident (integer)        |
| ident (writeln) | semicolon (;)          |
| lparent         | <identifier-list>      |
| ident (x)       | ident (ch)             |
| rparent         | colon (:)              |
| semicolon       | <type>                 |
| endsy           | ident (char)           |
| period          | semicolon (;)          |
|                 | <compound-statement>   |
|                 | beginsy (begin)        |
|                 | <statement-list>       |
|                 | <statement>            |
|                 | <assignment-statement> |
|                 | <variable>             |
|                 | ident (x)              |
|                 | becomes (:=)           |
|                 | <expression>           |
|                 | <simple-expression>    |
|                 | <term>                 |
|                 | <factor>               |
|                 | <variable>             |
|                 | ident (MAX)            |
|                 | semicolon (;)          |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <pre> &lt;statement&gt;   &lt;assignment-statement&gt;     &lt;variable&gt;       ident (ch)     becomes (:=)     &lt;expression&gt;       &lt;simple-expression&gt;         &lt;term&gt;           &lt;factor&gt;             &lt;variable&gt;               ident (INITIAL)           semicolon (;)         &lt;statement&gt;           &lt;procedure/function-call&gt;             ident (writeln)             lparent (())             &lt;parameter-list&gt;               &lt;expression&gt;                 &lt;simple-expression&gt;                   &lt;term&gt;                     &lt;factor&gt;                       &lt;variable&gt;                         ident (x)                     rparent ())             semicolon (;)           &lt;statement&gt;             &lt;empty&gt;           endsy (end)         period (.) </pre> |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <p>Pengujian kedua digunakan untuk memeriksa kemampuan parser dalam menangani deklarasi konstanta dan deklarasi tipe. Kasus ini mencakup konstanta integer, real, karakter, string, tipe range, serta tipe enumerasi. Output token menunjukkan bahwa simbol ==, tanda kurung, koma, dan konstanta berhasil dikenali dengan tipe token yang tepat. Parse tree yang terbentuk memperlihatkan bahwa parser dapat membangun node &lt;const-declaration&gt;, &lt;type-declaration&gt;, &lt;range&gt;, dan &lt;enumerated&gt; secara benar.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

### 3.3. Pengujian Ketiga

|                           |                                 |
|---------------------------|---------------------------------|
| <b>Input (input3.txt)</b> | <b>Output (parse_tree3.txt)</b> |
|---------------------------|---------------------------------|

```

MAR-Tubes-IF2224-2026 > test > milestone2 > input > E input3.txt
1  program ArrayAndRecord;
2  type
3      Matrix == array[1..3] of integer;
4      Person == record
5          name: string;
6          age: integer;
7          score: real;
8      end;
9  var
10     arr: array[1..5] of integer;
11     i: integer;
12     p: Person;
13 begin
14     arr[1] := 10;
15     arr[2] := 20;
16     arr[3] := arr[1] + arr[2];
17     p.age := 20;
18     p.score := 3.14;
19     for i := 1 to 5 do
20         writeln(arr[i]);
21     end.

```

## Output (token3.txt)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E token3.txt
1  programsy
2  ident (ArrayAndRecord)
3  semicolon
4  typesy
5  ident (Matrix)
6  eql
7  arraysy
8  lbrack
9  intcon (1)

```

programsy  
 ident (ArrayAndRecord)  
 semicolon  
 typesy  
 ident (Matrix)  
 eql  
 arraysy  
 lbrack  
 intcon (1)  
 period  
 period  
 intcon (3)  
 rbrack  
 ofsy  
 ident (integer)  
 semicolon  
 ident (Person)  
 eql  
 recordsy  
 ident (name)  
 colon  
 ident (string)  
 semicolon  
 ident (age)  
 colon  
 ident (integer)  
 semicolon  
 ident (score)  
 colon  
 ident (real)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E parse_tree3.txt
1  <program>
2  <program-header>
3  <programsy (program)>
4  <ident (ArrayAndRecord)>
5  <semicolon (;)>
6  <declaration-part>
7  <type-declaration>
8  <typesy (type)>
9  <type-definition>
10 <ident (Matrix)>
11 <eql (==)>
12 <type>

```

<program>  
 <program-header>  
 programsy (program)  
 ident (ArrayAndRecord)  
 semicolon (;)  
 <declaration-part>  
 <type-declaration>  
 typesy (type)  
 <type-definition>  
 ident (Matrix)  
 eql (==)  
 <type>  
 <array-type>  
 arraysy (array)  
 lbrack ([)  
 <range>  
 <constant>  
 intcon (1)  
 period (.)  
 period (.)  
 <constant>  
 intcon (3)  
 rbrack (])  
 ofsy (of)  
 <type>  
 ident (integer)  
 semicolon (;)  
 <type-definition>  
 ident (Person)  
 eql (==)  
 <type>  
 <record-type>  
 recordsy (record)  
 <field-list>  
 <field-part>  
 <identifier-list>  
 ident (name)  
 colon (:)  
 <type>  
 ident (string)  
 semicolon (;)  
 <field-part>

|                 |                      |
|-----------------|----------------------|
| endsy           | <identifier-list>    |
| semicolon       | ident (age)          |
| varsy           | colon (:)            |
| ident (arr)     | <type>               |
| colon           | ident (integer)      |
| arraysy         | semicolon (;)        |
| lbrack          | <field-part>         |
| intcon (1)      | <identifier-list>    |
| period          | ident (score)        |
| period          | colon (:)            |
| intcon (5)      | <type>               |
| rbrack          | ident (real)         |
| ofsy            | endsy (end)          |
| ident (integer) | semicolon (;)        |
| semicolon       | <var-declaration>    |
| ident (i)       | varsy (var)          |
| colon           | <identifier-list>    |
| ident (integer) | ident (arr)          |
| semicolon       | colon (:)            |
| ident (p)       | <type>               |
| colon           | <array-type>         |
| ident (Person)  | arraysy (array)      |
| semicolon       | lbrack (l)           |
| beginsy         | <range>              |
| ident (arr)     | <constant>           |
| lbrack          | intcon (1)           |
| intcon (1)      | period (.)           |
| rbrack          | period (.)           |
| becomes         | <constant>           |
| intcon (10)     | intcon (5)           |
| semicolon       | rbrack (r)           |
| ident (arr)     | ofsy (of)            |
| lbrack          | <type>               |
| intcon (2)      | ident (integer)      |
| rbrack          | semicolon (;)        |
| becomes         | <identifier-list>    |
| intcon (20)     | ident (i)            |
| semicolon       | colon (:)            |
| ident (arr)     | <type>               |
| lbrack          | ident (integer)      |
| intcon (3)      | semicolon (;)        |
| rbrack          | <identifier-list>    |
| becomes         | ident (p)            |
| ident (arr)     | colon (:)            |
| lbrack          | <type>               |
| intcon (1)      | ident (Person)       |
| rbrack          | semicolon (;)        |
| plus            | <compound-statement> |
| ident (arr)     | beginsy (begin)      |
| lbrack          | <statement-list>     |

|                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| intcon (2)<br>rbrack<br>semicolon<br>ident (p)<br>period<br>ident (age)<br>becomes<br>intcon (20)<br>semicolon<br>ident (p)<br>period<br>ident (score)<br>becomes<br>realcon (3.14)<br>semicolon<br>forsy<br>ident (i)<br>becomes<br>intcon (1)<br>tosy<br>intcon (5)<br>dosy<br>ident (writeln)<br>lparent<br>ident (arr)<br>lbrack<br>ident (i)<br>rbrack<br>rparent<br>semicolon<br>endsy<br>period | <statement><br><assignment-statement><br><variable><br>ident (arr)<br><selector><br>lbrack (]<br><index-list><br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>rbrack (]<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (10)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (arr)<br><selector><br>lbrack (]<br><index-list><br><expression><br><simple-expression><br><term><br><factor><br>intcon (2)<br>rbrack (]<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (20)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (arr)<br><selector><br>lbrack (]<br><index-list><br><expression><br><simple-expression><br><term> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>&lt;factor&gt;<br/>intcon (3)<br/>rbrack (])<br/>becomes (:=)<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (arr)<br/>&lt;selector&gt;<br/>lbrack ([)<br/>&lt;index-list&gt;<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (1)<br/>rbrack (])<br/>&lt;additive-operator&gt;<br/>plus (+)<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (arr)<br/>&lt;selector&gt;<br/>lbrack ([)<br/>&lt;index-list&gt;<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (2)<br/>rbrack (])<br/>semicolon (;)<br/>&lt;statement&gt;<br/>&lt;assignment-statement&gt;<br/>&lt;variable&gt;<br/>ident (p)<br/>&lt;selector&gt;<br/>period (.)<br/>ident (age)<br/>becomes (:=)<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (20)<br/>semicolon (;)<br/>&lt;statement&gt;</div> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <assignment-statement><br><variable><br>ident (p)<br><selector><br>period (.)<br>ident (score)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>realcon (3.14)<br>semicolon (;)<br><statement><br><for-statement><br>forsy (for)<br>ident (i)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>tosy (to)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (5)<br>dosy (do)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent (()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (arr)<br><selector><br>lbrack ([])<br><index-list><br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (i) |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | rbrack (])<br>rparent ())<br>semicolon (;)<br><statement><br><empty><br>endsy (end)<br>period (.) |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                   |
| Pengujian ketiga berfokus pada pemrosesan tipe data kompleks, yaitu array dan record. Program ini menguji deklarasi array, deklarasi record, pengaksesan elemen array, pengaksesan field record, dan perulangan for. Hasil token menunjukkan bahwa lexer mampu mengenali tanda kurung siku, operator titik, keyword array, record, of, dan token lain yang dibutuhkan. Parse tree yang dihasilkan membuktikan bahwa parser dapat membentuk struktur <array-type>, <record-type>, <variable>, serta selector untuk akses array dan field. |                                                                                                   |

### 3.4. Pengujian Keempat

| Input (input4.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Output (parse_tree4.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt;  input4.txt 1  program BranchingStatements; 2  var 3    x, y, z: integer; 4  begin 5    x := 10; 6    y := 20; 7    if x == y then 8      writeln('sama') 9    else begin 10     if x &gt; y then 11       z := x 12     else 13       z := y; 14     writeln(z) 15   end; 16   case x of 17     1: writeln('satu'); 18     2, 3: writeln('dua atau tiga'); 19     10: writeln('sepuluh') 20   end 21 end. </pre> | <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  parse_tree4.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4      ident (BranchingStatements) 5      semicolon (;) 6    &lt;declaration-part&gt; 7      &lt;var-declaration&gt; 8        varsy (var) 9        &lt;identifier-list&gt; 10         ident (x) 11         comma (,) 12         ident (y) 13         comma (,) 14         ident (z) </pre> |
| Output (token4.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  token4.txt 1  programsy 2  ident (BranchingStatements) 3  semicolon 4  varsy 5  ident (x) 6  comma 7  ident (y) 8  comma 9  ident (z) 10 colon 11 ident (integer) </pre>                                                                                                                                                                                                                                        | <pre> &lt;program&gt; &lt;program-header&gt;   programsy (program)   ident (BranchingStatements)   semicolon (;) &lt;declaration-part&gt;   &lt;var-declaration&gt;     varsy (var)     &lt;identifier-list&gt;       ident (x)       comma (,)       ident (y)       comma (,)       ident (z)     colon (:)   &lt;type&gt;     ident (integer)   semicolon (;) </pre>                                                                                       |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| varsy<br>ident (x)<br>comma<br>ident (y)<br>comma<br>ident (z)<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>ident (x)<br>becomes<br>intcon (10)<br>semicolon<br>ident (y)<br>becomes<br>intcon (20)<br>semicolon<br>ifsy<br>ident (x)<br>eql<br>ident (y)<br>thensy<br>ident (writeln)<br>lparent<br>string ('sama')<br>rparent<br>elsesy<br>beginsy<br>ifsy<br>ident (x)<br>gtr<br>ident (y)<br>thensy<br>ident (z)<br>becomes<br>ident (x)<br>elsesy<br>ident (z)<br>becomes<br>ident (y)<br>semicolon<br>ident (writeln)<br>lparent<br>ident (z)<br>rparent<br>endsy<br>semicolon<br>casesy<br>ident (x) | <compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><assignment-statement><br><variable><br>ident (x)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (10)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (y)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (20)<br>semicolon (;)<br><statement><br><if-statement><br>ifsy (if)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (x)<br><relational-operator><br>eql (==)<br><simple-expression><br><term><br><factor><br><variable><br>ident (y)<br>thensy (then)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ofsy<br>intcon (1)<br>colon<br>ident (writeln)<br>lparent<br>string ('satu')<br>rparent<br>semicolon<br>intcon (2)<br>comma<br>intcon (3)<br>colon<br>ident (writeln)<br>lparent<br>string ('dua atau tiga')<br>rparent<br>semicolon<br>intcon (10)<br>colon<br>ident (writeln)<br>lparent<br>string ('sepuluh')<br>rparent<br>endsy<br>endsy<br>period | <factor><br>string ('sama')<br>rparent ()<br>elsesy (else)<br><statement><br><compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><if-statement><br>ifsy (if)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (x)<br><relational-operator><br>gtr (>)<br><simple-expression><br><term><br><factor><br><variable><br>ident (y)<br>thensy (then)<br><statement><br><assignment-statement><br><variable><br>ident (z)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (x)<br>elsesy (else)<br><statement><br><assignment-statement><br><variable><br>ident (z)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (y)<br>semicolon (;)<br><statement> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre> &lt;procedure/function-call&gt;   ident (writeln)   lparent ()   &lt;parameter-list&gt;   &lt;expression&gt;     &lt;simple-expression&gt;     &lt;term&gt;       &lt;factor&gt;         &lt;variable&gt;           ident (z)         rparent ()       endsy (end)     semicolon (;)   &lt;statement&gt;     &lt;case-statement&gt;     casesy (case)     &lt;expression&gt;     &lt;simple-expression&gt;     &lt;term&gt;       &lt;factor&gt;         &lt;variable&gt;           ident (x)       ofsy (of)     &lt;case-block&gt;     &lt;constant&gt;     intcon (1)     colon (:)     &lt;statement&gt;     &lt;procedure/function-call&gt;     ident (writeln)     lparent ()     &lt;parameter-list&gt;     &lt;expression&gt;     &lt;simple-expression&gt;     &lt;term&gt;       &lt;factor&gt;         string ('satu')       rparent ()     semicolon (;)     &lt;case-block&gt;     &lt;constant&gt;     intcon (2)     comma (,)     &lt;constant&gt;     intcon (3)     colon (:)     &lt;statement&gt;     &lt;procedure/function-call&gt;     ident (writeln)     lparent () </pre> |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <pre> &lt;parameter-list&gt;   &lt;expression&gt;     &lt;simple-expression&gt;       &lt;term&gt;         &lt;factor&gt;           string ('dua atau tiga')         rparent ())       semicolon (;)     &lt;case-block&gt;       &lt;constant&gt;         intcon (10)       colon (:)     &lt;statement&gt;       &lt;procedure/function-call&gt;         ident (writeln)         lparent (())         &lt;parameter-list&gt;           &lt;expression&gt;             &lt;simple-expression&gt;               &lt;term&gt;                 &lt;factor&gt;                   string ('sepuluh')                 rparent ())           endsy (end)         endsy (end)       period (.) </pre> |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <p>Pengujian keempat digunakan untuk menguji struktur percabangan dalam bahasa Arion. Program ini memuat statement if-then-else, nested if, compound statement di dalam cabang else, serta case statement dengan beberapa pilihan nilai. Output token memperlihatkan bahwa operator relasional seperti == dan &gt; berhasil dikenali bersama keyword percabangan seperti if, then, else, case, dan of. Parse tree yang dihasilkan menunjukkan bahwa parser mampu membangun node &lt;if-statement&gt; dan &lt;case-statement&gt; secara bertingkat sesuai struktur input.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

### 3.5. Pengujian Kelima

|                           |                                 |
|---------------------------|---------------------------------|
| <b>Input (input5.txt)</b> | <b>Output (parse_tree5.txt)</b> |
|---------------------------|---------------------------------|

```

MAR-Tubes-IF2224-2026 > test > milestone2 > input >  input5.txt
1  program LoopStatements;
2  var
3    i, sum, factorial: integer;
4  begin
5    sum := 0;
6    i := 1;
7    while i <= 10 do begin
8      sum := sum + i;
9      i := i + 1
10   end;
11   factorial := 1;
12   i := 5;
13   repeat
14     factorial := factorial * i;
15     i := i - 1
16   until i == 0;
17   for i := 10 downto 1 do
18     writeln(i);
19     writeln(sum);
20     writeln(factorial)
21   end.

```

## Output (token5.txt)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output >  token5.txt
1  programsy
2  ident (LoopStatements)
3  semicolon
4  varsy
5  ident (i)
6  comma
7  ident (sum)
8  comma
9  ident (factorial)
10 colon
11 ident (integer)
12 semicolon
13 beginsy
14 ident (sum)
15 becomes

```

```

programsy
ident (LoopStatements)
semicolon
varsy
ident (i)
comma
ident (sum)
comma
ident (factorial)
colon
ident (integer)
semicolon
beginsy
ident (sum)
becomes
intcon (0)
semicolon
ident (i)
becomes
intcon (1)
semicolon
whilesy
ident (i)
leq
intcon (10)
dosy
beginsy
ident (sum)

```

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output >  parse_tree5.txt
1  <program>
2  <program-header>
3    programsy (program)
4    ident (LoopStatements)
5    semicolon (;)
6  <declaration-part>
7    <var-declaration>
8      varsy (var)
9      <identifier-list>
10        ident (i)
11        comma (,)
12        ident (sum)
13        comma (,)
14        ident (factorial)

```

```

<program>
<program-header>
  programsy (program)
  ident (LoopStatements)
  semicolon (;)
<declaration-part>
  <var-declaration>
    varsy (var)
    <identifier-list>
      ident (i)
      comma (,)
      ident (sum)
      comma (,)
      ident (factorial)
  colon (:)
  <type>
    ident (integer)
    semicolon (;)
  <compound-statement>
    beginsy (begin)
    <statement-list>
      <statement>
        <assignment-statement>
          <variable>
            ident (sum)
          becomes (:=)
          <expression>
            <simple-expression>
              <term>
                <factor>
                  intcon (0)
            semicolon (;)
          <statement>
            <assignment-statement>
              <variable>
                ident (i)
              becomes (:=)
              <expression>
                <simple-expression>
                  <term>
                    <factor>

```



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| becomes<br>ident (sum)<br>plus<br>ident (i)<br>semicolon<br>ident (i)<br>becomes<br>ident (i)<br>plus<br>intcon (1)<br>endsy<br>semicolon<br>ident (factorial)<br>becomes<br>intcon (1)<br>semicolon<br>ident (i)<br>becomes<br>intcon (5)<br>semicolon<br>repeatsy<br>ident (factorial)<br>becomes<br>ident (factorial)<br>times<br>ident (i)<br>semicolon<br>ident (i)<br>becomes<br>ident (i)<br>minus<br>intcon (1)<br>untilsy<br>ident (i)<br>eql<br>intcon (0)<br>semicolon<br>forsy<br>ident (i)<br>becomes<br>intcon (10)<br>downtosy<br>intcon (1)<br>dosy<br>ident (writeln)<br>lparent<br>ident (i)<br>rparent<br>semicolon<br>ident (writeln) | intcon (1)<br>semicolon (;)<br><statement><br><while-statement><br>whilesy (while)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (i)<br><relational-operator><br>leq (<=)<br><simple-expression><br><term><br><factor><br>intcon (10)<br>dosy (do)<br><statement><br><compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><assignment-statement><br><variable><br>ident (sum)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (sum)<br><additive-operator><br>plus (+)<br><term><br><factor><br><variable><br>ident (i)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (i)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                   |                           |
|-------------------|---------------------------|
| lparent           | ident (i)                 |
| ident (sum)       | <additive-operator>       |
| rparent           | plus (+)                  |
| semicolon         | <term>                    |
| ident (writeln)   | <factor>                  |
| lparent           | intcon (1)                |
| ident (factorial) | endsy (end)               |
| rparent           | semicolon (;)             |
| endsy             | <statement>               |
| period            | <assignment-statement>    |
|                   | <variable>                |
|                   | ident (factorial)         |
|                   | becomes (:=)              |
|                   | <expression>              |
|                   | <simple-expression>       |
|                   | <term>                    |
|                   | <factor>                  |
|                   | intcon (1)                |
|                   | semicolon (;)             |
|                   | <statement>               |
|                   | <assignment-statement>    |
|                   | <variable>                |
|                   | ident (i)                 |
|                   | becomes (:=)              |
|                   | <expression>              |
|                   | <simple-expression>       |
|                   | <term>                    |
|                   | <factor>                  |
|                   | intcon (5)                |
|                   | semicolon (;)             |
|                   | <statement>               |
|                   | <repeat-statement>        |
|                   | repeatsy (repeat)         |
|                   | <statement-list>          |
|                   | <statement>               |
|                   | <assignment-statement>    |
|                   | <variable>                |
|                   | ident (factorial)         |
|                   | becomes (:=)              |
|                   | <expression>              |
|                   | <simple-expression>       |
|                   | <term>                    |
|                   | <factor>                  |
|                   | <variable>                |
|                   | ident (factorial)         |
|                   | <multiplicative-operator> |
|                   | times (*)                 |
|                   | <factor>                  |
|                   | <variable>                |
|                   | ident (i)                 |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (i)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (i)<br><additive-operator><br>minus (-)<br><term><br><factor><br>intcon (1)<br>untilsy (until)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (i)<br><relational-operator><br>eql (==)<br><simple-expression><br><term><br><factor><br>intcon (0)<br>semicolon (;)<br><statement><br><for-statement><br>forsy (for)<br>ident (i)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (10)<br>downtosy (downto)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>dosy (do)<br><statement><br><procedure/function-call> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <pre> ident (writeln) lparent () &lt;parameter-list&gt; &lt;expression&gt; &lt;simple-expression&gt; &lt;term&gt; &lt;factor&gt; &lt;variable&gt; ident (i) rparent () semicolon (;) &lt;statement&gt; &lt;procedure/function-call&gt; ident (writeln) lparent () &lt;parameter-list&gt; &lt;expression&gt; &lt;simple-expression&gt; &lt;term&gt; &lt;factor&gt; &lt;variable&gt; ident (sum) rparent () semicolon (;) &lt;statement&gt; &lt;procedure/function-call&gt; ident (writeln) lparent () &lt;parameter-list&gt; &lt;expression&gt; &lt;simple-expression&gt; &lt;term&gt; &lt;factor&gt; &lt;variable&gt; ident (factorial) rparent () endsy (end) period (.) </pre> |
| <p><b>Keterangan</b></p> <p>Pengujian kelima bertujuan untuk memastikan parser dapat menangani berbagai bentuk perulangan. Program ini memuat while-do, repeat-until, dan for-downto dalam satu source code. Hasil token menunjukkan bahwa keyword perulangan serta operator seperti &lt;=, ==, *, dan - diklasifikasikan dengan benar. Parse tree yang terbentuk memperlihatkan bahwa parser berhasil mengenali &lt;while-statement&gt;, &lt;repeat-statement&gt;, dan &lt;for-statement&gt; beserta statement yang berada di dalamnya.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

### 3.6. Pengujian Keenam

| Input (input6.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Output (parse_tree6.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt;  input6.txt 1  program ProcedureAndFunction; 2  procedure Greet(name: string); 3  begin 4      writeln('Hello, ', name) 5    end; 6  procedure Swap(a, b: integer); 7  var 8      temp: integer; 9    begin program ProcedureAndFunction; procedure Greet(name: string); begin   writeln('Hello, ', name) end; procedure Swap(a, b: integer); var   temp: integer; begin   temp := a;   a := b;   b := temp end; function Max(a, b: integer): integer; begin   if a &gt; b then     Max := a   else     Max := b   end; function Factorial(n: integer): integer; begin   if n == 0 then     Factorial := 1   else     Factorial := n * Factorial(n - 1)   end; var   x, y: integer; begin   x := 5;   y := 3;   Greet('World');   Swap(x, y);   writeln(Max(x, y));   writeln(Factorial(5)) end. </pre> | <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  parse_tree6.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4      ident (ProcedureAndFunction) 5      semicolon (;) 6    &lt;declaration-part&gt; 7      &lt;subprogram-declaration-part&gt; 8        &lt;subprogram-declaration&gt; 9          &lt;procedure-declaration&gt; 10           &lt;procedure-heading&gt; 11           proceduresy (procedure) 12           ident (Greet) 13           &lt;formal-parameter-list&gt; 14           lparent (()) 15           &lt;parameter-group&gt; 16           &lt;identifier-list&gt; &lt;program&gt; &lt;program-header&gt;   programsy (program)   ident (ProcedureAndFunction)   semicolon (;) &lt;declaration-part&gt;   &lt;subprogram-declaration-part&gt;     &lt;subprogram-declaration&gt;       &lt;procedure-declaration&gt;         &lt;procedure-heading&gt;           proceduresy (procedure)           ident (Greet)           &lt;formal-parameter-list&gt;             lparent (())             &lt;parameter-group&gt;               &lt;identifier-list&gt;                 ident (name)                 colon (:)                 ident (string)             rparent (())           semicolon (;)         &lt;block&gt;           &lt;declaration-part&gt;             &lt;compound-statement&gt;               beginsy (begin)                 &lt;statement-list&gt;                   &lt;statement&gt;                     &lt;procedure/function-call&gt;                       ident (writeln)                       lparent (())                       &lt;parameter-list&gt;                         &lt;expression&gt;                           &lt;simple-expression&gt;                             &lt;term&gt;                               &lt;factor&gt;                                 string ('Hello, ') </pre> |
| Output (token6.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

MAR-Tubes-IF2224-2026 > test > milestone2 > output > token6.txt

```
1  programsy
2  ident (ProcedureAndFunction)
3  semicolon
4  proceduressy
5  ident (Greet)
6  lparent
7  ident (name)
8  colon
9  ident (string)
10 rparent
11 semicolon
12 beginsy
13 ident (writeln)
```

programsy  
ident (ProcedureAndFunction)  
semicolon  
proceduressy  
ident (Greet)  
lparent  
ident (name)  
colon  
ident (string)  
rparent  
semicolon  
beginsy  
ident (writeln)  
lparent  
string ('Hello, ')  
comma  
ident (name)  
rparent  
endsy  
semicolon  
proceduressy  
ident (Swap)  
lparent  
ident (a)  
comma  
ident (b)  
colon  
ident (integer)  
rparent  
semicolon  
varsy  
ident (temp)  
colon  
ident (integer)  
semicolon  
beginsy  
ident (temp)  
becomes  
ident (a)  
semicolon  
ident (a)  
becomes

comma (,)  
<expression>  
<simple-expression>  
<term>  
<factor>  
<variable>  
ident (name)  
rparent ())  
endsy (end)  
semicolon (;)  
<subprogram-declaration>  
<procedure-declaration>  
<procedure-heading>  
proceduressy (procedure)  
ident (Swap)  
<formal-parameter-list>  
lparent ()  
<parameter-group>  
<identifier-list>  
ident (a)  
comma (,)  
ident (b)  
colon (:)  
ident (integer)  
rparent ())  
semicolon (;)  
<block>  
<declaration-part>  
<var-declaration>  
varsy (var)  
<identifier-list>  
ident (temp)  
colon (:)  
<type>  
ident (integer)  
semicolon (;)  
<compound-statement>  
beginsy (begin)  
<statement-list>  
<statement>  
<assignment-statement>  
<variable>  
ident (temp)  
becomes (:=)  
<expression>  
<simple-expression>  
<term>  
<factor>  
<variable>  
ident (a)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ident (b)<br>semicolon<br>ident (b)<br>becomes<br>ident (temp)<br>endsy<br>semicolon<br>functionsy<br>ident (Max)<br>lparent<br>ident (a)<br>comma<br>ident (b)<br>colon<br>ident (integer)<br>rparent<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>ifsy<br>ident (a)<br>gtr<br>ident (b)<br>thensy<br>ident (Max)<br>becomes<br>ident (a)<br>elsesy<br>ident (Max)<br>becomes<br>ident (b)<br>endsy<br>semicolon<br>functionsy<br>ident (Factorial)<br>lparent<br>ident (n)<br>colon<br>ident (integer)<br>rparent<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>ifsy<br>ident (n)<br>eq<br>intcon (0)<br>thensy | semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (a)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (b)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (b)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (temp)<br>endsy (end)<br>semicolon (;)<br><subprogram-declaration><br><function-declaration><br><function-heading><br>functionsy (function)<br>ident (Max)<br><formal-parameter-list><br>lparent ()<br><parameter-group><br><identifier-list><br>ident (a)<br>comma (,)<br>ident (b)<br>colon (:)<br>ident (integer)<br>rparent ())<br>colon (:)<br>ident (integer)<br>semicolon (;)<br><block><br><declaration-part><br><compound-statement><br>beginsy (begin)<br><statement-list><br><statement> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ident (Factorial)<br>becomes<br>intcon (1)<br>elsesy<br>ident (Factorial)<br>becomes<br>ident (n)<br>times<br>ident (Factorial)<br>lparent<br>ident (n)<br>minus<br>intcon (1)<br>rparent<br>endsy<br>semicolon<br>varsy<br>ident (x)<br>comma<br>ident (y)<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>ident (x)<br>becomes<br>intcon (5)<br>semicolon<br>ident (y)<br>becomes<br>intcon (3)<br>semicolon<br>ident (Greet)<br>lparent<br>string ('World')<br>rparent<br>semicolon<br>ident (Swap)<br>lparent<br>ident (x)<br>comma<br>ident (y)<br>rparent<br>semicolon<br>ident (writeln)<br>lparent<br>ident (Max)<br>lparent<br>ident (x)<br>comma | <if-statement><br>ifsy (if)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a)<br><relational-operator><br>gtr (>)<br><simple-expression><br><term><br><factor><br><variable><br>ident (b)<br>thensy (then)<br><statement><br><assignment-statement><br><variable><br>ident (Max)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a)<br>elsesy (else)<br><statement><br><assignment-statement><br><variable><br>ident (Max)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (b)<br>endsy (end)<br>semicolon (;)<br><subprogram-declaration><br><function-declaration><br><function-heading><br>functionsy (function)<br>ident (Factorial)<br><formal-parameter-list><br>lparent ()<br><parameter-group><br><identifier-list> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p> ident (y)<br/> rparent<br/> rparent<br/> semicolon<br/> ident (writeln)<br/> lparent<br/> ident (Factorial)<br/> lparent<br/> intcon (5)<br/> rparent<br/> rparent<br/> endsy<br/> period </p> | <p> ident (n)<br/> colon (:)<br/> ident (integer)<br/> rparent ())<br/> colon (:)<br/> ident (integer)<br/> semicolon (;)<br/> &lt;block&gt;<br/> &lt;declaration-part&gt;<br/> &lt;compound-statement&gt;<br/> beginsy (begin)<br/> &lt;statement-list&gt;<br/> &lt;statement&gt;<br/> &lt;if-statement&gt;<br/> ifsy (if)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (n)<br/> &lt;relational-operator&gt;<br/> eql (==)<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> intcon (0)<br/> thensy (then)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (Factorial)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> intcon (1)<br/> elsesy (else)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (Factorial)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (n) </p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | $\langle \text{multiplicative-operator} \rangle$<br>times (*)<br>$\langle \text{factor} \rangle$<br>$\langle \text{procedure/function-call} \rangle$<br>ident (Factorial)<br>lparent ()<br>$\langle \text{parameter-list} \rangle$<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (n)<br>$\langle \text{additive-operator} \rangle$<br>minus (-)<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (1)<br>rparent ()<br>endsy (end)<br>semicolon (;)<br>$\langle \text{var-declaration} \rangle$<br>varsy (var)<br>$\langle \text{identifier-list} \rangle$<br>ident (x)<br>comma (,)<br>ident (y)<br>colon (:)<br>$\langle \text{type} \rangle$<br>ident (integer)<br>semicolon (;)<br>$\langle \text{compound-statement} \rangle$<br>beginsy (begin)<br>$\langle \text{statement-list} \rangle$<br>$\langle \text{statement} \rangle$<br>$\langle \text{assignment-statement} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (x)<br>becomes (:=)<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (5)<br>semicolon (;)<br>$\langle \text{statement} \rangle$<br>$\langle \text{assignment-statement} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (y)<br>becomes (:=) |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```

    <expression>
      <simple-expression>
        <term>
          <factor>
            intcon (3)
semicolon (;)
<statement>
  <procedure/function-call>
    ident (Greet)
    lparent ()
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              string ('World')
    rparent ()
semicolon (;)
<statement>
  <procedure/function-call>
    ident (Swap)
    lparent ()
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              <variable>
                ident (x)
      comma (,)
      <expression>
        <simple-expression>
          <term>
            <factor>
              <variable>
                ident (y)
    rparent ()
semicolon (;)
<statement>
  <procedure/function-call>
    ident (writeln)
    lparent ()
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              <procedure/function-call>
                ident (Max)
            lparent ()

```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre> &lt;parameter-list&gt;   &lt;expression&gt;     &lt;simple-expression&gt;       &lt;term&gt;         &lt;factor&gt;           &lt;variable&gt;             ident (x)         comma (,)       &lt;expression&gt;         &lt;simple-expression&gt;           &lt;term&gt;             &lt;factor&gt;               &lt;variable&gt;                 ident (y)           rparent ())         rparent ()       semicolon (;)     &lt;statement&gt;       &lt;procedure/function-call&gt;         ident (writeln)         lparent ()         &lt;parameter-list&gt;           &lt;expression&gt;             &lt;simple-expression&gt;               &lt;term&gt;                 &lt;factor&gt;                   &lt;procedure/function-call&gt;                     ident (Factorial)                     lparent ()                     &lt;parameter-list&gt;                       &lt;expression&gt;                         &lt;simple-expression&gt;                           &lt;term&gt;                             &lt;factor&gt;                               intcon (5)                         rparent ())                     rparent ()                   rparent ()                 endsy (end)               period (.) </pre> |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Keterangan

Pengujian keenam difokuskan pada deklarasi dan pemanggilan subprogram. Program ini mencakup deklarasi prosedur, deklarasi fungsi, parameter formal, variabel lokal, pemanggilan prosedur, pemanggilan fungsi, dan rekursi pada fungsi Factorial. Output token menunjukkan bahwa lexer mampu mengenali token seperti proceduressy, functionsy, lparent, rparent, colon, dan token ekspresi lain yang digunakan dalam subprogram. Parse tree membuktikan bahwa parser dapat membentuk node <procedure-declaration>, <function-declaration>, <formal-parameter-list>, dan <procedure/function-call> dengan struktur yang sesuai.

### 3.7. Pengujian Ketujuh

| Input (input7.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Output (parse_tree7.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt;  input7.txt 1  program Expressions; 2  var 3    a, b, c: integer; 4    x, y: real; 5    p, q, r: boolean; 6  begin 7    a := 10; 8    b := 3; 9    c := a + b * 2 - 1; 10   c := a div b; 11   c := a mod b; 12   x := 10.0 / 3.0; 13   c := -a + b; 14   p := a == b; 15   q := a &lt;&gt; b; 16   r := (a &gt; 0) AND (b &gt; 0); 17   r := (a &gt; 0) OR (b &lt; 0); 18   r := NOT (a == b); 19   if (a &gt; b) AND NOT (a == 0) then 20     writeln(a) 21   end. </pre>                                                   | <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  parse_tree7.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4        ident (Expressions) 5        semicolon (;) 6      &lt;declaration-part&gt; 7        &lt;var-declaration&gt; 8          varsy (var) 9            &lt;identifier-list&gt; 10              ident (a) 11              comma (,) 12              ident (b) 13              comma (,) 14              ident (c) 15              colon (:) 16              &lt;type&gt; 17                ident (integer) </pre>                                                                                           |
| Output (token7.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <pre> MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  token7.txt 1  programsy 2  ident (Expressions) 3  semicolon 4  varsy 5  ident (a) 6  comma 7  ident (b) 8  comma 9  ident (c) 10 colon 11 ident (integer) 12 semicolon 13 ident (x) </pre> <p> programsy<br/> ident (Expressions)<br/> semicolon<br/> varsy<br/> ident (a)<br/> comma<br/> ident (b)<br/> comma<br/> ident (c)<br/> colon<br/> ident (integer)<br/> semicolon<br/> ident (x)<br/> comma<br/> ident (y)<br/> colon<br/> ident (real)<br/> semicolon<br/> ident (p)<br/> comma<br/> ident (q)<br/> comma </p> | <pre> &lt;program&gt; &lt;program-header&gt;   programsy (program)   ident (Expressions)   semicolon (;) &lt;declaration-part&gt;   &lt;var-declaration&gt;     varsy (var)     &lt;identifier-list&gt;       ident (a)       comma (,)       ident (b)       comma (,)       ident (c)       colon (:)     &lt;type&gt;       ident (integer)     semicolon (;)   &lt;identifier-list&gt;     ident (x)     comma (,)     ident (y)     colon (:)   &lt;type&gt;     ident (real)     semicolon (;)   &lt;identifier-list&gt;     ident (p)     comma (,)     ident (q)     comma (,)     ident (r)     colon (:)   &lt;type&gt;     ident (boolean)     semicolon (;) </pre> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ident (r)<br>colon<br>ident (boolean)<br>semicolon<br>beginsy<br>ident (a)<br>becomes<br>intcon (10)<br>semicolon<br>ident (b)<br>becomes<br>intcon (3)<br>semicolon<br>ident (c)<br>becomes<br>ident (a)<br>plus<br>ident (b)<br>times<br>intcon (2)<br>minus<br>intcon (1)<br>semicolon<br>ident (c)<br>becomes<br>ident (a)<br>idiv<br>ident (b)<br>semicolon<br>ident (c)<br>becomes<br>ident (a)<br>imod<br>ident (b)<br>semicolon<br>ident (x)<br>becomes<br>realcon (10.0)<br>rdiv<br>realcon (3.0)<br>semicolon<br>ident (c)<br>becomes<br>minus<br>ident (a)<br>plus<br>ident (b)<br>semicolon<br>ident (p)<br>becomes | <compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><assignment-statement><br><variable><br>ident (a)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (10)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (b)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (3)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (c)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a)<br><additive-operator><br>plus (+)<br><term><br><factor><br><variable><br>ident (b)<br><multiplicative-operator><br>times (*)<br><factor><br>intcon (2)<br><additive-operator><br>minus (-)<br><term><br><factor> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p> ident (a)<br/> eql<br/> ident (b)<br/> semicolon<br/> ident (q)<br/> becomes<br/> ident (a)<br/> neq<br/> ident (b)<br/> semicolon<br/> ident (r)<br/> becomes<br/> lparent<br/> ident (a)<br/> gtr<br/> intcon (0)<br/> rparent<br/> andsy<br/> lparent<br/> ident (b)<br/> gtr<br/> intcon (0)<br/> rparent<br/> semicolon<br/> ident (r)<br/> becomes<br/> lparent<br/> ident (a)<br/> gtr<br/> intcon (0)<br/> rparent<br/> orsy<br/> lparent<br/> ident (b)<br/> lss<br/> intcon (0)<br/> rparent<br/> semicolon<br/> ident (r)<br/> becomes<br/> notsy<br/> lparent<br/> ident (a)<br/> eql<br/> ident (b)<br/> rparent<br/> semicolon<br/> ifsy<br/> lparent<br/> ident (a) </p> | <p> intcon (1)<br/> semicolon (;)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (c)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (a)<br/> &lt;multiplicative-operator&gt;<br/> idiv (div)<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (b)<br/> semicolon (;)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (c)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (a)<br/> &lt;multiplicative-operator&gt;<br/> imod (mod)<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (b)<br/> semicolon (;)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (x)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> realcon (10.0)<br/> &lt;multiplicative-operator&gt;<br/> rdiv (/)<br/> &lt;factor&gt;<br/> realcon (3.0) </p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gtr<br>ident (b)<br>rparent<br>andsy<br>notsy<br>lparent<br>ident (a)<br>eql<br>intcon (0)<br>rparent<br>thensy<br>ident (writeln)<br>lparent<br>ident (a)<br>rparent<br>endsy<br>period | semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (c)<br>becomes (:=)<br><expression><br><simple-expression><br>minus (-)<br><term><br><factor><br><variable><br>ident (a)<br><additive-operator><br>plus (+)<br><term><br><factor><br><variable><br>ident (b)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (p)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a)<br><relational-operator><br>eql (==)<br><simple-expression><br><term><br><factor><br><variable><br>ident (b)<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (q)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a) |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>&lt;relational-operator&gt;<br/>neq (&lt; &gt;)<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (b)<br/>semicolon (;)<br/>&lt;statement&gt;<br/>&lt;assignment-statement&gt;<br/>&lt;variable&gt;<br/>ident (r)<br/>becomes (:=)<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>lparent (<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (a)<br/>&lt;relational-operator&gt;<br/>gtr (&gt;)<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (0)<br/>rparent ( )<br/>&lt;multiplicative-operator&gt;<br/>andsy (AND)<br/>&lt;factor&gt;<br/>lparent (<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (b)<br/>&lt;relational-operator&gt;<br/>gtr (&gt;)<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (0)<br/>rparent ( )<br/>semicolon (;)<br/>&lt;statement&gt;</div> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <assignment-statement><br><variable><br>ident (r)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>lparent ()<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (a)<br><relational-operator><br>gtr (>)<br><simple-expression><br><term><br><factor><br>intcon (0)<br>rparent ())<br><additive-operator><br>orsy (OR)<br><term><br><factor><br>lparent ()<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (b)<br><relational-operator><br>lss (<)<br><simple-expression><br><term><br><factor><br>intcon (0)<br>rparent ())<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (r)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>notsy (NOT)</div> <div>&lt;factor&gt;</div> <div>lparent (()</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (a)</div> <div>&lt;relational-operator&gt;</div> <div>eql (==)</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (b)</div> <div>rparent ())</div> <div>semicolon (;)</div> <div>&lt;statement&gt;</div> <div>&lt;if-statement&gt;</div> <div>ifsy (if)</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>lparent (()</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (a)</div> <div>&lt;relational-operator&gt;</div> <div>gtr (&gt;)</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (b)</div> <div>rparent ())</div> <div>&lt;multiplicative-operator&gt;</div> <div>andsy (AND)</div> <div>&lt;factor&gt;</div> <div>notsy (NOT)</div> <div>&lt;factor&gt;</div> <div>lparent (()</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <pre> &lt;variable&gt;   ident (a) &lt;relational-operator&gt;   eql (==) &lt;simple-expression&gt;   &lt;term&gt;     &lt;factor&gt;       intcon (0)     rparent ()   thensy (then) &lt;statement&gt;   &lt;procedure/function-call&gt;     ident (writeln)     lparent ()     &lt;parameter-list&gt;       &lt;expression&gt;         &lt;simple-expression&gt;           &lt;term&gt;             &lt;factor&gt;               &lt;variable&gt;                 ident (a)             rparent ()           endsy (end)         period (.) </pre> |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <p>Pengujian ketujuh digunakan untuk memvalidasi parsing ekspresi yang lebih kompleks. Program ini mencakup operasi aritmatika, pembagian integer, pembagian real, modulo, unary minus, operator relasional, serta operator logika AND, OR, dan NOT. Output token menunjukkan bahwa operator dengan prioritas berbeda berhasil dikenali sebagai token yang sesuai, seperti times, idiv, rdiv, imod, andsy, orsy, dan notsy. Parse tree menunjukkan bahwa parser berhasil memisahkan struktur &lt;expression&gt;, &lt;simple-expression&gt;, &lt;term&gt;, dan &lt;factor&gt; sehingga prioritas operator dapat direpresentasikan dengan jelas.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

### 3.8. Pengujian Kedelapan

|                           |                                 |
|---------------------------|---------------------------------|
| <b>Input (input8.txt)</b> | <b>Output (parse_tree8.txt)</b> |
|---------------------------|---------------------------------|

```

MAR-Tubes-IF2224-2026 > test > milestone2 > input > E input8.txt
1 program CommentsAndConstants;
2 { Ini adalah program untuk menguji komentar dan konstanta }
3 const
4   MAXSIZE == 50;
5   MINVAL == -10;
6   LABEL == 'Data';
7 var
8   i: integer;
9   ch: char;
10  s: string;
11 begin
12  { komentar di dalam begin }
13  i := MAXSIZE;
14  (* ini juga komentar
15   * multi baris *)
16  ch := 'Z';
17  s := 'Hello World';
18  i := MINVAL + 5;
19  writeln(i);
20  writeln(ch);
21  writeln(s);
22 end.

```

## Output (token8.txt)

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E token8.txt
1 programsy
2 ident (CommentsAndConstants)
3 semicolon
4 comment ({ Ini adalah program untuk menguji komentar dan konstanta })
5 constsy
6 ident (MAXSIZE)
7 eql
8 intcon (50)
9 semicolon
10 ident (MINVAL)
11 eql
12 minus

```

programsy  
 ident (CommentsAndConstants)  
 semicolon  
 comment ( { Ini adalah program untuk menguji  
 komentar dan konstanta } )  
 constsy  
 ident (MAXSIZE)  
 eql  
 intcon (50)  
 semicolon  
 ident (MINVAL)  
 eql  
 minus  
 intcon (10)  
 semicolon  
 ident (LABEL)  
 eql  
 string ('Data')  
 semicolon  
 varsy  
 ident (i)  
 colon  
 ident (integer)  
 semicolon  
 ident (ch)  
 colon  
 ident (char)  
 semicolon  
 ident (s)  
 colon

```

MAR-Tubes-IF2224-2026 > test > milestone2 > output > E parse_tree8.txt
1 <program>
2   <program-header>
3     programsy (program)
4       ident (CommentsAndConstants)
5       semicolon (;)
6   <declaration-part>
7     <const-declaration>
8       constsy (const)
9       <const-definition>
10        ident (MAXSIZE)
11        eql (==)
12        <constant>
13        intcon (50)
14        semicolon (;)

```

<program>  
 <program-header>  
 programsy (program)  
 ident (CommentsAndConstants)  
 semicolon (;)  
 <declaration-part>  
 <const-declaration>  
 constsy (const)  
 <const-definition>  
 ident (MAXSIZE)  
 eql (==)  
 <constant>  
 intcon (50)  
 semicolon (;)  
 <const-definition>  
 ident (MINVAL)  
 eql (==)  
 <constant>  
 minus (-)  
 intcon (10)  
 semicolon (;)  
 <const-definition>  
 ident (LABEL)  
 eql (==)  
 <constant>  
 string ('Data')  
 semicolon (;)  
 <var-declaration>  
 varsy (var)  
 <identifier-list>  
 ident (i)  
 colon (:)  
 <type>  
 ident (integer)  
 semicolon (;)  
 <identifier-list>  
 ident (ch)  
 colon (:)  
 <type>  
 ident (char)  
 semicolon (;)  
 <identifier-list>

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p> ident (string)<br/> semicolon<br/> beginsy<br/> comment ( { komentar di dalam begin } )<br/> ident (i)<br/> becomes<br/> ident (MAXSIZE)<br/> semicolon<br/> comment (( * ini juga komentar<br/> multi baris * ))<br/> ident (ch)<br/> becomes<br/> charcon ('Z')<br/> semicolon<br/> ident (s)<br/> becomes<br/> string ('Hello World')<br/> semicolon<br/> ident (i)<br/> becomes<br/> ident (MINVAL)<br/> plus<br/> intcon (5)<br/> semicolon<br/> ident (writeln)<br/> lparent<br/> ident (i)<br/> rparent<br/> semicolon<br/> ident (writeln)<br/> lparent<br/> ident (ch)<br/> rparent<br/> semicolon<br/> ident (writeln)<br/> lparent<br/> ident (s)<br/> rparent<br/> endsy<br/> period </p> | <p> ident (s)<br/> colon (:)<br/> &lt;type&gt;<br/> ident (string)<br/> semicolon (;)<br/> &lt;compound-statement&gt;<br/> beginsy (begin)<br/> &lt;statement-list&gt;<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (i)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> &lt;variable&gt;<br/> ident (MAXSIZE)<br/> semicolon (;)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (ch)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> charcon ('Z')<br/> semicolon (;)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (s)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt;<br/> &lt;factor&gt;<br/> string ('Hello World')<br/> semicolon (;)<br/> &lt;statement&gt;<br/> &lt;assignment-statement&gt;<br/> &lt;variable&gt;<br/> ident (i)<br/> becomes (:=)<br/> &lt;expression&gt;<br/> &lt;simple-expression&gt;<br/> &lt;term&gt; </p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```

    <factor>
    <variable>
    ident (MINVAL)
    <additive-operator>
    plus (+)
    <term>
    <factor>
    intcon (5)
semicolon (;)
<statement>
    <procedure/function-call>
    ident (writeln)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (i)
    rparent ()
semicolon (;)
<statement>
    <procedure/function-call>
    ident (writeln)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (ch)
    rparent ()
semicolon (;)
<statement>
    <procedure/function-call>
    ident (writeln)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (s)
    rparent ()
endsy (end)
period (.)

```

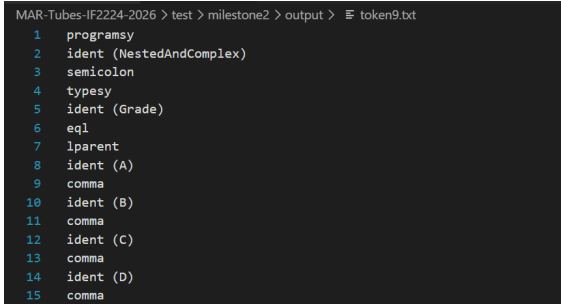
### Keterangan

Pengujian kedelapan berfokus pada penanganan komentar dan konstanta. Program ini memuat komentar dengan format { ... } dan (\* ... \*), serta konstanta bernilai positif, negatif, karakter, dan string. Output token menunjukkan bahwa komentar berhasil dikenali oleh lexer sebagai token comment, sedangkan parse tree tidak memasukkan komentar ke dalam struktur sintaks utama karena komentar tidak memengaruhi grammar program. Hasil ini menunjukkan bahwa sistem mampu membedakan elemen yang relevan untuk parsing dari elemen yang hanya bersifat anotatif pada source code.

## 3.9. Pengujian Kesembilan

| Input (input9.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Output (parse_tree9.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt; E input9.txt 1  program NestedAndComplex; 2  type 3    Grade == (A, B, C, D, E); 4    ScoreRange == 0..100; 5  var 6    score, i, j: integer; 7    grade: Grade; 8    matrix: array[1..3] of integer; 9    procedure PrintMatrix(size: integer); 10   var 11     k: integer; 12   begin 13     for k := 1 to size do 14       writeln(matrix[k]) 15   end;</pre> <pre>program NestedAndComplex; type   Grade == (A, B, C, D, E);   ScoreRange == 0..100; var   score, i, j: integer;   grade: Grade;   matrix: array[1..3] of integer; procedure PrintMatrix(size: integer); var   k: integer; begin   for k := 1 to size do     writeln(matrix[k]) end; function GetGrade(s: integer): integer; begin   if s &gt;= 80 then     GetGrade := 1   else if s &gt;= 60 then     GetGrade := 2   else     GetGrade := 3 end; begin   for i := 1 to 3 do begin</pre> | <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt; E parse_tree9.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4        ident (NestedAndComplex) 5        semicolon (;) 6      &lt;declaration-part&gt; 7        &lt;type-declaration&gt; 8          typesy (type) 9          &lt;type-definition&gt; 10             ident (Grade) 11             eql (==) 12             &lt;type&gt; 13               &lt;enumerated&gt;</pre> <pre>&lt;program&gt; &lt;program-header&gt;   programsy (program)   ident (NestedAndComplex)   semicolon (;) &lt;declaration-part&gt;   &lt;type-declaration&gt;     typesy (type)   &lt;type-definition&gt;     ident (Grade)     eql (==)   &lt;type&gt;     &lt;enumerated&gt;     lparent (()     &lt;identifier-list&gt;       ident (A)       comma (,)       ident (B)       comma (,)       ident (C)       comma (,)       ident (D)       comma (,)       ident (E)     rparent ())     semicolon (;)   &lt;type-definition&gt;</pre> |



|                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> matrix[i] := i * 10; end; PrintMatrix(3); score := 75; case GetGrade(score) of   1: writeln('A');   2: writeln('B');   3: writeln('C') end; i := 1; while i &lt;= 3 do begin   j := 1;   while j &lt;= 3 do begin     writeln(i * j);     j := j + 1   end;   i := i + 1 end end. </pre> | <pre> ident (ScoreRange) eq1 (==) &lt;type&gt;   &lt;range&gt;     &lt;constant&gt;       intcon (0)     period (.)     period (.)     &lt;constant&gt;       intcon (100) semicolon (;) &lt;var-declaration&gt; varsy (var) &lt;identifier-list&gt;   ident (score)   comma (,)   ident (i)   comma (,)   ident (j) colon (:) &lt;type&gt;   ident (integer) semicolon (;) &lt;identifier-list&gt;   ident (grade) colon (:) &lt;type&gt;   ident (Grade) semicolon (;) &lt;identifier-list&gt;   ident (matrix) colon (:) &lt;type&gt;   &lt;array-type&gt;     arraysy (array)     lbrack ([])     &lt;range&gt;       &lt;constant&gt;         intcon (1)       period (.)       period (.)       &lt;constant&gt;         intcon (3)     rbrack (])     ofsy (of)     &lt;type&gt;       ident (integer) semicolon (;) &lt;subprogram-declaration-part&gt;   &lt;subprogram-declaration&gt; </pre> |
| <p><b>Output (token9.txt)</b></p>                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|  <pre> programsy ident (NestedAndComplex) semicolon typesy ident (Grade) eq1 lparent ident (A) comma ident (B) comma ident (C) comma ident (D) comma ident (E) rparent semicolon ident (ScoreRange) </pre>   | <pre> ident (integer) semicolon (;) &lt;identifier-list&gt;   ident (grade) colon (:) &lt;type&gt;   ident (Grade) semicolon (;) &lt;identifier-list&gt;   ident (matrix) colon (:) &lt;type&gt;   &lt;array-type&gt;     arraysy (array)     lbrack ([])     &lt;range&gt;       &lt;constant&gt;         intcon (1)       period (.)       period (.)       &lt;constant&gt;         intcon (3)     rbrack (])     ofsy (of)     &lt;type&gt;       ident (integer) semicolon (;) &lt;subprogram-declaration-part&gt;   &lt;subprogram-declaration&gt; </pre>                                                                                                                                                                                                                                                                                                                                         |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| eql<br>intcon (0)<br>period<br>period<br>intcon (100)<br>semicolon<br>varsy<br>ident (score)<br>comma<br>ident (i)<br>comma<br>ident (j)<br>colon<br>ident (integer)<br>semicolon<br>ident (grade)<br>colon<br>ident (Grade)<br>semicolon<br>ident (matrix)<br>colon<br>arraysy<br>lbrack<br>intcon (1)<br>period<br>period<br>intcon (3)<br>rbrack<br>ofsy<br>ident (integer)<br>semicolon<br>proceduresy<br>ident (PrintMatrix)<br>lparent<br>ident (size)<br>colon<br>ident (integer)<br>rparent<br>semicolon<br>varsy<br>ident (k)<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>forsy<br>ident (k)<br>becomes<br>intcon (1)<br>tosy | <procedure-declaration><br><procedure-heading><br>proceduresy (procedure)<br>ident (PrintMatrix)<br><formal-parameter-list><br>lparent ()<br><parameter-group><br><identifier-list><br>ident (size)<br>colon (:)<br>ident (integer)<br>rparent ()<br>semicolon (;)<br><block><br><declaration-part><br><var-declaration><br>varsy (var)<br><identifier-list><br>ident (k)<br>colon (:)<br><type><br>ident (integer)<br>semicolon (;)<br><compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><for-statement><br>forsy (for)<br>ident (k)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>tosy (to)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (size)<br>dosy (do)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent ()<br><parameter-list><br><expression> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                  |                          |
|------------------|--------------------------|
| ident (size)     | <simple-expression>      |
| dosy             | <term>                   |
| ident (writeln)  | <factor>                 |
| lparent          | <variable>               |
| ident (matrix)   | ident (matrix)           |
| lbrack           | <selector>               |
| ident (k)        | lbrack (]                |
| rbrack           | <index-list>             |
| rparent          | <expression>             |
| endsy            | <simple-expression>      |
| semicolon        | <term>                   |
| functionsy       | <factor>                 |
| ident (GetGrade) | <variable>               |
| lparent          | ident (k)                |
| ident (s)        | rbrack (]                |
| colon            | rparent ())              |
| ident (integer)  | endsy (end)              |
| rparent          | semicolon (;)            |
| colon            | <subprogram-declaration> |
| ident (integer)  | <function-declaration>   |
| semicolon        | <function-heading>       |
| beginsy          | functionsy (function)    |
| ifsy             | ident (GetGrade)         |
| ident (s)        | <formal-parameter-list>  |
| geq              | lparent (()              |
| intcon (80)      | <parameter-group>        |
| thensy           | <identifier-list>        |
| ident (GetGrade) | ident (s)                |
| becomes          | colon (:)                |
| intcon (1)       | ident (integer)          |
| elsesy           | rparent ())              |
| ifsy             | colon (:)                |
| ident (s)        | ident (integer)          |
| geq              | semicolon (;)            |
| intcon (60)      | <block>                  |
| thensy           | <declaration-part>       |
| ident (GetGrade) | <compound-statement>     |
| becomes          | beginsy (begin)          |
| intcon (2)       | <statement-list>         |
| elsesy           | <statement>              |
| ident (GetGrade) | <if-statement>           |
| becomes          | ifsy (if)                |
| intcon (3)       | <expression>             |
| endsy            | <simple-expression>      |
| semicolon        | <term>                   |
| beginsy          | <factor>                 |
| forsy            | <variable>               |
| ident (i)        | ident (s)                |
| becomes          | <relational-operator>    |
| intcon (1)       | geq (>=)                 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tosy<br>intcon (3)<br>dosy<br>beginsy<br>ident (matrix)<br>lbrack<br>ident (i)<br>rbrack<br>becomes<br>ident (i)<br>times<br>intcon (10)<br>semicolon<br>endsy<br>semicolon<br>ident (PrintMatrix)<br>lparent<br>intcon (3)<br>rparent<br>semicolon<br>ident (score)<br>becomes<br>intcon (75)<br>semicolon<br>casesy<br>ident (GetGrade)<br>lparent<br>ident (score)<br>rparent<br>ofsy<br>intcon (1)<br>colon<br>ident (writeln)<br>lparent<br>charcon ('A')<br>rparent<br>semicolon<br>intcon (2)<br>colon<br>ident (writeln)<br>lparent<br>charcon ('B')<br>rparent<br>semicolon<br>intcon (3)<br>colon<br>ident (writeln)<br>lparent<br>charcon ('C')<br>rparent | <simple-expression><br><term><br><factor><br>intcon (80)<br>thensy (then)<br><statement><br><assignment-statement><br><variable><br>ident (GetGrade)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>elsesy (else)<br><statement><br><if-statement><br>ifsy (if)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (s)<br><relational-operator><br>geq (>=)<br><simple-expression><br><term><br><factor><br>intcon (60)<br>thensy (then)<br><statement><br><assignment-statement><br><variable><br>ident (GetGrade)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (2)<br>elsesy (else)<br><statement><br><assignment-statement><br><variable><br>ident (GetGrade)<br>becomes (:=)<br><expression><br><simple-expression> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                 |                        |
|-----------------|------------------------|
| endsy           | <term>                 |
| semicolon       | <factor>               |
| ident (i)       | intcon (3)             |
| becomes         | endsy (end)            |
| intcon (1)      | semicolon (;)          |
| semicolon       | <compound-statement>   |
| whilesy         | beginsy (begin)        |
| ident (i)       | <statement-list>       |
| leq             | <statement>            |
| intcon (3)      | <for-statement>        |
| dosy            | forsy (for)            |
| beginsy         | ident (i)              |
| ident (j)       | becomes (:=)           |
| becomes         | <expression>           |
| intcon (1)      | <simple-expression>    |
| semicolon       | <term>                 |
| whilesy         | <factor>               |
| ident (j)       | intcon (1)             |
| leq             | tosy (to)              |
| intcon (3)      | <expression>           |
| dosy            | <simple-expression>    |
| beginsy         | <term>                 |
| ident (writeln) | <factor>               |
| lparent         | intcon (3)             |
| ident (i)       | dosy (do)              |
| times           | <statement>            |
| ident (j)       | <compound-statement>   |
| rparent         | beginsy (begin)        |
| semicolon       | <statement-list>       |
| ident (j)       | <statement>            |
| becomes         | <assignment-statement> |
| ident (j)       | <variable>             |
| plus            | ident (matrix)         |
| intcon (1)      | <selector>             |
| endsy           | lbrack ([)             |
| semicolon       | <index-list>           |
| ident (i)       | <expression>           |
| becomes         | <simple-expression>    |
| ident (i)       | <term>                 |
| plus            | <factor>               |
| intcon (1)      | <variable>             |
| endsy           | ident (i)              |
| endsy           | rbrack (])             |
| period          | becomes (:=)           |
|                 | <expression>           |
|                 | <simple-expression>    |
|                 | <term>                 |
|                 | <factor>               |
|                 | <variable>             |
|                 | ident (i)              |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <multiplicative-operator><br>times (*)<br><factor><br>intcon (10)<br>semicolon (;)<br><statement><br><empty><br>endsy (end)<br>semicolon (;)<br><statement><br><procedure/function-call><br>ident (PrintMatrix)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br>intcon (3)<br>rparent ()<br>semicolon (;)<br><statement><br><assignment-statement><br><variable><br>ident (score)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (75)<br>semicolon (;)<br><statement><br><case-statement><br>casesy (case)<br><expression><br><simple-expression><br><term><br><factor><br><procedure/function-call><br>ident (GetGrade)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (score)<br>rparent () |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | ofsy (of)<br><case-block><br><constant><br>intcon (1)<br>colon (:)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br>charcon ('A')<br>rparent ()<br>semicolon (;)<br><case-block><br><constant><br>intcon (2)<br>colon (:)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br>charcon ('B')<br>rparent ()<br>semicolon (;)<br><case-block><br><constant><br>intcon (3)<br>colon (:)<br><statement><br><procedure/function-call><br>ident (writeln)<br>lparent ()<br><parameter-list><br><expression><br><simple-expression><br><term><br><factor><br>charcon ('C')<br>rparent ()<br>endsy (end)<br>semicolon (;) |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <statement><br><assignment-statement><br><variable><br>ident (i)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>semicolon (;)<br><statement><br><while-statement><br>whilesy (while)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (i)<br><relational-operator><br>leq (<=)<br><simple-expression><br><term><br><factor><br>intcon (3)<br>dosy (do)<br><statement><br><compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><assignment-statement><br><variable><br>ident (j)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>semicolon (;)<br><statement><br><while-statement><br>whilesy (while)<br><expression><br><simple-expression><br><term><br><factor><br><variable> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>ident (j)</div> <div>&lt;relational-operator&gt;</div> <div>leq (&lt;=)</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>intcon (3)</div> <div>dosy (do)</div> <div>&lt;statement&gt;</div> <div>&lt;compound-statement&gt;</div> <div>beginsy (begin)</div> <div>&lt;statement-list&gt;</div> <div>&lt;statement&gt;</div> <div>&lt;procedure/function-call&gt;</div> <div>ident (writeln)</div> <div>lparent (()</div> <div>&lt;parameter-list&gt;</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (i)</div> <div>&lt;multiplicative-operator&gt;</div> <div>times (*)</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (j)</div> <div>rparent ())</div> <div>semicolon (;)</div> <div>&lt;statement&gt;</div> <div>&lt;assignment-statement&gt;</div> <div>&lt;variable&gt;</div> <div>ident (j)</div> <div>becomes (:=)</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (j)</div> <div>&lt;additive-operator&gt;</div> <div>plus (+)</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>intcon (1)</div> <div>endsy (end)</div> <div>semicolon (;)</div> <div>&lt;statement&gt;</div> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <pre>&lt;assignment-statement&gt;   &lt;variable&gt;     ident (i)   becomes (:=)   &lt;expression&gt;     &lt;simple-expression&gt;       &lt;term&gt;         &lt;factor&gt;           &lt;variable&gt;             ident (i)         &lt;additive-operator&gt;           plus (+)       &lt;term&gt;         &lt;factor&gt;           intcon (1)     endsy (end)   endsy (end) period (.)</pre> |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                    |
| <p>Pengujian kesembilan menggabungkan beberapa fitur tingkat lanjut dalam satu program. Kasus ini mencakup deklarasi enumerasi, range, array, prosedur, fungsi, nested if, pemanggilan fungsi di dalam case, serta nested while. Output token menunjukkan bahwa seluruh komponen program kompleks dapat dipecah menjadi deretan token yang lengkap dan berurutan. Parse tree yang dihasilkan memperlihatkan bahwa parser mampu menjaga hierarki struktur program, terutama pada bagian subprogram, statement bersarang, dan ekspresi yang digunakan sebagai parameter.</p> |                                                                                                                                                                                                                                                                                                                                                                                                    |

3.10. Pengujian Kesepuluh

| Input (input10.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Output (parse_tree10.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; input &gt; F input10.txt 1  program FullFeatures; 2  const 3    N == 5; 4    TRUEVAL == 1; 5  type 6    Arr == array[1..5] of integer; 7    Direction == (North, South, East, West); 8    Coord == record 9      x: integer; 10     y: integer; 11   end; 12   var 13     nums: Arr; 14     pos: Coord; 15     dir: Direction; 16     i, j, temp, total: integer; 17   procedure BubbleSort(size: integer);</pre> <p>program FullFeatures;<br/>const<br/>  N == 5;<br/>  TRUEVAL == 1;<br/>type<br/>  Arr == array[1..5] of integer;<br/>  Direction == (North, South, East, West);<br/>  Coord == record</p> | <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt; F parse_tree10.txt 1  &lt;program&gt; 2    &lt;program-header&gt; 3      programsy (program) 4      ident (FullFeatures) 5      semicolon (;) 6    &lt;declaration-part&gt; 7      &lt;const-declaration&gt; 8        constay (const) 9        &lt;const-definition&gt; 10         ident (N) 11         eql (==) 12         &lt;constant&gt; 13         intcon (5) 14       semicolon (;) 15      &lt;const-definition&gt; 16       ident (TRUEVAL) 17       eql (==)</pre> <p>&lt;program&gt;<br/>&lt;program-header&gt;<br/>  programsy (program)<br/>  ident (FullFeatures)<br/>  semicolon (;)<br/>&lt;declaration-part&gt;</p> |

```

    x: integer;
    y: integer
end;
var
    nums: Arr;
    pos: Coord;
    dir: Direction;
    i, j, temp, total: integer;
procedure BubbleSort(size: integer);
var
    k, l: integer;
begin
    for k := 1 to size - 1 do
        for l := 1 to size - k do
            if nums[l] > nums[l + 1] then begin
                temp := nums[l];
                nums[l] := nums[l + 1];
                nums[l + 1] := temp
            end
        end
    end;
end;
function Sum(size: integer): integer;
var
    k, s: integer;
begin
    s := 0;
    for k := 1 to size do
        s := s + nums[k];
    Sum := s
end;
begin
    nums[1] := 64;
    nums[2] := 34;
    nums[3] := 25;
    nums[4] := 12;
    nums[5] := 22;
    BubbleSort(N);
    total := Sum(N);
    for i := 1 to N do
        writeln(nums[i]);
    writeln(total);
    pos.x := 0;
    pos.y := 0;
    i := 1;
    repeat
        if i mod 2 == 0 then
            pos.x := pos.x + 1
        else
            pos.y := pos.y + 1;
        i := i + 1
    until i > 10;

```

```

<const-declaration>
constsy (const)
<const-definition>
ident (N)
eq1 (==)
<constant>
intcon (5)
semicolon (;)
<const-definition>
ident (TRUEVAL)
eq1 (==)
<constant>
intcon (1)
semicolon (;)
<type-declaration>
typesy (type)
<type-definition>
ident (Arr)
eq1 (==)
<type>
<array-type>
arraysy (array)
lbrack ([])
<range>
<constant>
intcon (1)
period (.)
period (.)
<constant>
intcon (5)
rbrack ([])
ofsy (of)
<type>
ident (integer)
semicolon (;)
<type-definition>
ident (Direction)
eq1 (==)
<type>
<enumerated>
lparent (())
<identifier-list>
ident (North)
comma (,)
ident (South)
comma (,)
ident (East)
comma (,)
ident (West)
rparent (())

```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>writeln(pos.x); writeln(pos.y) end.</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <p><b>Output (token10.txt)</b></p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <div data-bbox="280 405 833 678"> <pre>MAR-Tubes-IF2224-2026 &gt; test &gt; milestone2 &gt; output &gt;  token10.txt 1  programsy 2  ident (FullFeatures) 3  semicolon 4  constsy 5  ident (N) 6  eql 7  intcon (5) 8  semicolon 9  ident (TRUEVAL) 10 eql 11 intcon (1) 12 semicolon 13 typesy 14 ident (Arr)</pre> </div> <pre> programsy ident (FullFeatures) semicolon constsy ident (N) eql intcon (5) semicolon ident (TRUEVAL) eql intcon (1) semicolon typesy ident (Arr) eql arraysy lbrack intcon (1) period period intcon (5) rbrack ofsy ident (integer) semicolon ident (Direction) eql lparent ident (North) comma ident (South) comma ident (East) comma ident (West) rparent </pre> | <pre> semicolon (;) &lt;type-definition&gt; ident (Coord) eql (==) &lt;type&gt; &lt;record-type&gt; recordsy (record) &lt;field-list&gt; &lt;field-part&gt; &lt;identifier-list&gt; ident (x) colon (:) &lt;type&gt; ident (integer) semicolon (;) &lt;field-part&gt; &lt;identifier-list&gt; ident (y) colon (:) &lt;type&gt; ident (integer) endsy (end) semicolon (;) &lt;var-declaration&gt; varsy (var) &lt;identifier-list&gt; ident (nums) colon (:) &lt;type&gt; ident (Arr) semicolon (;) &lt;identifier-list&gt; ident (pos) colon (:) &lt;type&gt; ident (Coord) semicolon (;) &lt;identifier-list&gt; ident (dir) colon (:) &lt;type&gt; ident (Direction) semicolon (;) &lt;identifier-list&gt; ident (i) comma (,) ident (j) comma (,) ident (temp) comma (,) </pre> |

|                    |                               |
|--------------------|-------------------------------|
| semicolon          | ident (total)                 |
| ident (Coord)      | colon (:                      |
| eql                | <type>                        |
| recordsy           | ident (integer)               |
| ident (x)          | semicolon (;)                 |
| colon              | <subprogram-declaration-part> |
| ident (integer)    | <subprogram-declaration>      |
| semicolon          | <procedure-declaration>       |
| ident (y)          | <procedure-heading>           |
| colon              | proceduresy (procedure)       |
| ident (integer)    | ident (BubbleSort)            |
| endsy              | <formal-parameter-list>       |
| semicolon          | lparent (()                   |
| varsy              | <parameter-group>             |
| ident (nums)       | <identifier-list>             |
| colon              | ident (size)                  |
| ident (Arr)        | colon (:                      |
| semicolon          | ident (integer)               |
| ident (pos)        | rparent ())                   |
| colon              | semicolon (;)                 |
| ident (Coord)      | <block>                       |
| semicolon          | <declaration-part>            |
| ident (dir)        | <var-declaration>             |
| colon              | varsy (var)                   |
| ident (Direction)  | <identifier-list>             |
| semicolon          | ident (k)                     |
| ident (i)          | comma (,                      |
| comma              | ident (l)                     |
| ident (j)          | colon (:                      |
| comma              | <type>                        |
| ident (temp)       | ident (integer)               |
| comma              | semicolon (;)                 |
| ident (total)      | <compound-statement>          |
| colon              | beginsy (begin)               |
| ident (integer)    | <statement-list>              |
| semicolon          | <statement>                   |
| proceduresy        | <for-statement>               |
| ident (BubbleSort) | forsy (for)                   |
| lparent            | ident (k)                     |
| ident (size)       | becomes (:=)                  |
| colon              | <expression>                  |
| ident (integer)    | <simple-expression>           |
| rparent            | <term>                        |
| semicolon          | <factor>                      |
| varsy              | intcon (1)                    |
| ident (k)          | tosy (to)                     |
| comma              | <expression>                  |
| ident (l)          | <simple-expression>           |
| colon              | <term>                        |
| ident (integer)    | <factor>                      |

|              |                     |
|--------------|---------------------|
| semicolon    | <variable>          |
| beginsy      | ident (size)        |
| forsy        | <additive-operator> |
| ident (k)    | minus (-)           |
| becomes      | <term>              |
| intcon (1)   | <factor>            |
| tosy         | intcon (1)          |
| ident (size) | dosy (do)           |
| minus        | <statement>         |
| intcon (1)   | <for-statement>     |
| dosy         | forsy (for)         |
| forsy        | ident (l)           |
| ident (l)    | becomes (:=)        |
| becomes      | <expression>        |
| intcon (1)   | <simple-expression> |
| tosy         | <term>              |
| ident (size) | <factor>            |
| minus        | intcon (1)          |
| ident (k)    | tosy (to)           |
| dosy         | <expression>        |
| ifsy         | <simple-expression> |
| ident (nums) | <term>              |
| lbrack       | <factor>            |
| ident (l)    | <variable>          |
| rbrack       | ident (size)        |
| gtr          | <additive-operator> |
| ident (nums) | minus (-)           |
| lbrack       | <term>              |
| ident (l)    | <factor>            |
| plus         | <variable>          |
| intcon (1)   | ident (k)           |
| rbrack       | dosy (do)           |
| thensy       | <statement>         |
| beginsy      | <if-statement>      |
| ident (temp) | ifsy (if)           |
| becomes      | <expression>        |
| ident (nums) | <simple-expression> |
| lbrack       | <term>              |
| ident (l)    | <factor>            |
| rbrack       | <variable>          |
| semicolon    | ident (nums)        |
| ident (nums) | <selector>          |
| lbrack       | lbrack (l)          |
| ident (l)    | <index-list>        |
| rbrack       | <expression>        |
| becomes      |                     |
| ident (nums) | <simple-expression> |
| lbrack       |                     |
| ident (l)    | <term>              |
| plus         | <factor>            |
|              | <variable>          |

|                 |                        |
|-----------------|------------------------|
| intcon (1)      | ident (l)              |
| rbrack          | rbrack (])             |
| semicolon       | <relational-operator>  |
| ident (nums)    | gtr (>)                |
| lbrack          | <simple-expression>    |
| ident (l)       | <term>                 |
| plus            | <factor>               |
| intcon (1)      | <variable>             |
| rbrack          | ident (nums)           |
| becomes         | <selector>             |
| ident (temp)    | lbrack ([)             |
| endsy           | <index-list>           |
| endsy           | <expression>           |
| semicolon       |                        |
| functionsy      | <simple-expression>    |
| ident (Sum)     | <term>                 |
| lparent         | <factor>               |
| ident (size)    | <variable>             |
| colon           | ident (l)              |
| ident (integer) |                        |
| rparent         | <additive-operator>    |
| colon           |                        |
| ident (integer) | plus (+)               |
| semicolon       | <term>                 |
| varsy           | <factor>               |
| ident (k)       | intcon (1)             |
| comma           | rbrack (])             |
| ident (s)       | thensy (then)          |
| colon           | <statement>            |
| ident (integer) | <compound-statement>   |
| semicolon       | beginsy (begin)        |
| beginsy         | <statement-list>       |
| ident (s)       | <statement>            |
| becomes         | <assignment-statement> |
| intcon (0)      | <variable>             |
| semicolon       | ident (temp)           |
| forsy           | becomes (:=)           |
| ident (k)       | <expression>           |
| becomes         | <simple-expression>    |
| intcon (1)      | <term>                 |
| tosy            | <factor>               |
| ident (size)    | <variable>             |
| dosy            | ident (nums)           |
| ident (s)       | <selector>             |
| becomes         | lbrack ([)             |
| ident (s)       | <index-list>           |
| plus            | <expression>           |
| ident (nums)    |                        |
| lbrack          | <simple-expression>    |
| ident (k)       | <term>                 |
|                 | <factor>               |

|                    |                        |
|--------------------|------------------------|
| rbrack             |                        |
| semicolon          |                        |
| ident (Sum)        |                        |
| becomes            |                        |
| ident (s)          |                        |
| endsy              |                        |
| semicolon          |                        |
| beginsy            |                        |
| ident (nums)       |                        |
| lbrack             |                        |
| intcon (1)         |                        |
| rbrack             |                        |
| becomes            |                        |
| intcon (64)        |                        |
| semicolon          |                        |
| ident (nums)       |                        |
| lbrack             |                        |
| intcon (2)         |                        |
| rbrack             |                        |
| becomes            |                        |
| intcon (34)        |                        |
| semicolon          |                        |
| ident (nums)       |                        |
| lbrack             |                        |
| intcon (3)         |                        |
| rbrack             |                        |
| becomes            |                        |
| intcon (25)        |                        |
| semicolon          |                        |
| ident (nums)       |                        |
| lbrack             |                        |
| intcon (4)         |                        |
| rbrack             |                        |
| becomes            |                        |
| intcon (12)        |                        |
| semicolon          |                        |
| ident (nums)       |                        |
| lbrack             |                        |
| intcon (5)         |                        |
| rbrack             |                        |
| becomes            |                        |
| intcon (22)        |                        |
| semicolon          |                        |
| ident (BubbleSort) |                        |
| lparent            |                        |
| ident (N)          |                        |
| rparent            |                        |
| semicolon          |                        |
| ident (total)      |                        |
| becomes            |                        |
|                    | <variable>             |
|                    | ident (l)              |
|                    | rbrack (l)             |
|                    | semicolon (;)          |
|                    | <statement>            |
|                    | <assignment-statement> |
|                    | <variable>             |
|                    | ident (nums)           |
|                    | <selector>             |
|                    | lbrack (l)             |
|                    | <index-list>           |
|                    | <expression>           |
|                    | <simple-expression>    |
|                    | <term>                 |
|                    | <factor>               |
|                    | <variable>             |
|                    | ident (l)              |
|                    | rbrack (l)             |
|                    | becomes (:=)           |
|                    | <expression>           |
|                    | <simple-expression>    |
|                    | <term>                 |
|                    | <factor>               |
|                    | <variable>             |
|                    | ident (nums)           |
|                    | <selector>             |
|                    | lbrack (l)             |
|                    | <index-list>           |
|                    | <expression>           |
|                    | <simple-expression>    |
|                    | <term>                 |
|                    | <factor>               |
|                    | <variable>             |
|                    | ident (l)              |
|                    | <additive-operator>    |
|                    | plus (+)               |
|                    | <term>                 |
|                    | <factor>               |
|                    | intcon (1)             |
|                    | rbrack (l)             |
|                    | semicolon (;)          |
|                    | <statement>            |
|                    | <assignment-statement> |
|                    | <variable>             |
|                    | ident (nums)           |



|                 |                          |
|-----------------|--------------------------|
| ident (Sum)     | <selector>               |
| lparent         | lbrack (l)               |
| ident (N)       | <index-list>             |
| rparent         | <expression>             |
| semicolon       |                          |
| forsy           | <simple-expression>      |
| ident (i)       | <term>                   |
| becomes         | <factor>                 |
| intcon (1)      | <variable>               |
| tosy            | ident (l)                |
| ident (N)       |                          |
| dosy            | <additive-operator>      |
| ident (writeln) | plus (+)                 |
| lparent         | <term>                   |
| ident (nums)    | <factor>                 |
| lbrack          | intcon (1)               |
| ident (i)       | rbrack (r)               |
| rbrack          | becomes (:=)             |
| rparent         | <expression>             |
| semicolon       | <simple-expression>      |
| ident (writeln) | <term>                   |
| lparent         | <factor>                 |
| ident (total)   | <variable>               |
| rparent         | ident (temp)             |
| semicolon       | endsy (end)              |
| ident (pos)     | endsy (end)              |
| period          | semicolon (;)            |
| ident (x)       | <subprogram-declaration> |
| becomes         | <function-declaration>   |
| intcon (0)      | <function-heading>       |
| semicolon       | functionsy (function)    |
| ident (pos)     | ident (Sum)              |
| period          | <formal-parameter-list>  |
| ident (y)       | lparent (l)              |
| becomes         | <parameter-group>        |
| intcon (0)      | <identifier-list>        |
| semicolon       | ident (size)             |
| ident (i)       | colon (:)                |
| becomes         | ident (integer)          |
| intcon (1)      | rparent (r)              |
| semicolon       | colon (:)                |
| repeatsy        | ident (integer)          |
| ifsy            | semicolon (;)            |
| ident (i)       | <block>                  |
| imod            | <declaration-part>       |
| intcon (2)      | <var-declaration>        |
| eq              | varsy (var)              |
| intcon (0)      | <identifier-list>        |
| thensy          | ident (k)                |
| ident (pos)     | comma (,)                |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| period<br>ident (x)<br>becomes<br>ident (pos)<br>period<br>ident (x)<br>plus<br>intcon (1)<br>elsesy<br>ident (pos)<br>period<br>ident (y)<br>becomes<br>ident (pos)<br>period<br>ident (y)<br>plus<br>intcon (1)<br>semicolon<br>ident (i)<br>becomes<br>ident (i)<br>plus<br>intcon (1)<br>untilsy<br>ident (i)<br>gtr<br>intcon (10)<br>semicolon<br>ident (writeln)<br>lparent<br>ident (pos)<br>period<br>ident (x)<br>rparent<br>semicolon<br>ident (writeln)<br>lparent<br>ident (pos)<br>period<br>ident (y)<br>rparent<br>endsy<br>period | ident (s)<br>colon (:)<br><type><br>ident (integer)<br>semicolon (;)<br><compound-statement><br>beginsy (begin)<br><statement-list><br><statement><br><assignment-statement><br><variable><br>ident (s)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (0)<br>semicolon (;)<br><statement><br><for-statement><br>forsy (for)<br>ident (k)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br>intcon (1)<br>tosy (to)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (size)<br>dosy (do)<br><statement><br><assignment-statement><br><variable><br>ident (s)<br>becomes (:=)<br><expression><br><simple-expression><br><term><br><factor><br><variable><br>ident (s)<br><additive-operator><br>plus (+) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | $\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (nums)<br>$\langle \text{selector} \rangle$<br>lbrack ( [ )<br>$\langle \text{index-list} \rangle$<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (k)<br>rbrack ( ] )<br>semicolon ( ; )<br>$\langle \text{statement} \rangle$<br>$\langle \text{assignment-statement} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (Sum)<br>becomes ( := )<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (s)<br>endsy (end)<br>semicolon ( ; )<br>$\langle \text{compound-statement} \rangle$<br>beginsy (begin)<br>$\langle \text{statement-list} \rangle$<br>$\langle \text{statement} \rangle$<br>$\langle \text{assignment-statement} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (nums)<br>$\langle \text{selector} \rangle$<br>lbrack ( [ )<br>$\langle \text{index-list} \rangle$<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (1)<br>rbrack ( ] )<br>becomes ( := )<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (64) |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre> semicolon (;) &lt;statement&gt;   &lt;assignment-statement&gt;     &lt;variable&gt;       ident (nums)     &lt;selector&gt;       lbrack ([])     &lt;index-list&gt;     &lt;expression&gt;       &lt;simple-expression&gt;         &lt;term&gt;           &lt;factor&gt;             intcon (2)           rbrack (])         becomes (:=)       &lt;expression&gt;         &lt;simple-expression&gt;           &lt;term&gt;             &lt;factor&gt;               intcon (34)           semicolon (;)         &lt;statement&gt;           &lt;assignment-statement&gt;             &lt;variable&gt;               ident (nums)             &lt;selector&gt;               lbrack ([])             &lt;index-list&gt;             &lt;expression&gt;               &lt;simple-expression&gt;                 &lt;term&gt;                   &lt;factor&gt;                     intcon (3)                 rbrack (])               becomes (:=)             &lt;expression&gt;               &lt;simple-expression&gt;                 &lt;term&gt;                   &lt;factor&gt;                     intcon (25)           semicolon (;)         &lt;statement&gt;           &lt;assignment-statement&gt;             &lt;variable&gt;               ident (nums)             &lt;selector&gt;               lbrack ([])             &lt;index-list&gt;             &lt;expression&gt;               &lt;simple-expression&gt; </pre> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | $\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (4)<br>rbrack (])<br>becomes ( $:=$ )<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (12)<br>semicolon (;)<br>$\langle \text{statement} \rangle$<br>$\langle \text{assignment-statement} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (nums)<br>$\langle \text{selector} \rangle$<br>lbrack ([)<br>$\langle \text{index-list} \rangle$<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (5)<br>rbrack (])<br>becomes ( $:=$ )<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>intcon (22)<br>semicolon (;)<br>$\langle \text{statement} \rangle$<br>$\langle \text{procedure/function-call} \rangle$<br>ident (BubbleSort)<br>lparent ((<br>$\langle \text{parameter-list} \rangle$<br>$\langle \text{expression} \rangle$<br>$\langle \text{simple-expression} \rangle$<br>$\langle \text{term} \rangle$<br>$\langle \text{factor} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (N)<br>rparent ())<br>semicolon (;)<br>$\langle \text{statement} \rangle$<br>$\langle \text{assignment-statement} \rangle$<br>$\langle \text{variable} \rangle$<br>ident (total)<br>becomes ( $:=$ )<br>$\langle \text{expression} \rangle$ |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```

    <simple-expression>
    <term>
    <factor>
    <procedure/function-call>
    ident (Sum)
    lparent ( ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (N)
    rparent ())
semicolon (;)
<statement>
<for-statement>
forsy (for)
ident (i)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
intcon (1)
tosy (to)
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (N)
dosy (do)
<statement>
<procedure/function-call>
ident (writeln)
lparent ( ()
<parameter-list>
<expression>
<simple-expression>
<term>
<factor>
<variable>
ident (nums)
<selector>
lbrack ( [
<index-list>
<expression>
<simple-expression>
<term>

```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (i)<br/>rbrack (])<br/>rparent ())<br/>semicolon (;)<br/>&lt;statement&gt;<br/>&lt;procedure/function-call&gt;<br/>ident (writeln)<br/>lparent (<br/>&lt;parameter-list&gt;<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>&lt;variable&gt;<br/>ident (total)<br/>rparent ())<br/>semicolon (;)<br/>&lt;statement&gt;<br/>&lt;assignment-statement&gt;<br/>&lt;variable&gt;<br/>ident (pos)<br/>&lt;selector&gt;<br/>period (.)<br/>ident (x)<br/>becomes (:=)<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (0)<br/>semicolon (;)<br/>&lt;statement&gt;<br/>&lt;assignment-statement&gt;<br/>&lt;variable&gt;<br/>ident (pos)<br/>&lt;selector&gt;<br/>period (.)<br/>ident (y)<br/>becomes (:=)<br/>&lt;expression&gt;<br/>&lt;simple-expression&gt;<br/>&lt;term&gt;<br/>&lt;factor&gt;<br/>intcon (0)<br/>semicolon (;)<br/>&lt;statement&gt;<br/>&lt;assignment-statement&gt;<br/>&lt;variable&gt;</div> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>             ident (i)<br/>             becomes (:=)<br/>             &lt;expression&gt;<br/>             &lt;simple-expression&gt;<br/>             &lt;term&gt;<br/>             &lt;factor&gt;<br/>             intcon (1)<br/>             semicolon (;)<br/>             &lt;statement&gt;<br/>             &lt;repeat-statement&gt;<br/>             repeatsy (repeat)<br/>             &lt;statement-list&gt;<br/>             &lt;statement&gt;<br/>             &lt;if-statement&gt;<br/>             ifsy (if)<br/>             &lt;expression&gt;<br/>             &lt;simple-expression&gt;<br/>             &lt;term&gt;<br/>             &lt;factor&gt;<br/>             &lt;variable&gt;<br/>             ident (i)<br/>             &lt;multiplicative-operator&gt;<br/>             imod (mod)<br/>             &lt;factor&gt;<br/>             intcon (2)<br/>             &lt;relational-operator&gt;<br/>             eql (==)<br/>             &lt;simple-expression&gt;<br/>             &lt;term&gt;<br/>             &lt;factor&gt;<br/>             intcon (0)<br/>             thensy (then)<br/>             &lt;statement&gt;<br/>             &lt;assignment-statement&gt;<br/>             &lt;variable&gt;<br/>             ident (pos)<br/>             &lt;selector&gt;<br/>             period (.)<br/>             ident (x)<br/>             becomes (:=)<br/>             &lt;expression&gt;<br/>             &lt;simple-expression&gt;<br/>             &lt;term&gt;<br/>             &lt;factor&gt;<br/>             &lt;variable&gt;<br/>             ident (pos)<br/>             &lt;selector&gt;<br/>             period (.)<br/>             ident (x)<br/>             &lt;additive-operator&gt; </p> |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>plus (+)</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>intcon (1)</div> <div>elsesy (else)</div> <div>&lt;statement&gt;</div> <div>&lt;assignment-statement&gt;</div> <div>&lt;variable&gt;</div> <div>ident (pos)</div> <div>&lt;selector&gt;</div> <div>period (.)</div> <div>ident (y)</div> <div>becomes (:=)</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (pos)</div> <div>&lt;selector&gt;</div> <div>period (.)</div> <div>ident (y)</div> <div>&lt;additive-operator&gt;</div> <div>plus (+)</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>intcon (1)</div> <div>semicolon (;)</div> <div>&lt;statement&gt;</div> <div>&lt;assignment-statement&gt;</div> <div>&lt;variable&gt;</div> <div>ident (i)</div> <div>becomes (:=)</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> <div>ident (i)</div> <div>&lt;additive-operator&gt;</div> <div>plus (+)</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>intcon (1)</div> <div>untilsy (until)</div> <div>&lt;expression&gt;</div> <div>&lt;simple-expression&gt;</div> <div>&lt;term&gt;</div> <div>&lt;factor&gt;</div> <div>&lt;variable&gt;</div> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre>             ident (i)             &lt;relational-operator&gt;             gtr (&gt;)             &lt;simple-expression&gt;             &lt;term&gt;             &lt;factor&gt;             intcon (10)         semicolon (;)         &lt;statement&gt;         &lt;procedure/function-call&gt;         ident (writeln)         lparent (())         &lt;parameter-list&gt;         &lt;expression&gt;         &lt;simple-expression&gt;         &lt;term&gt;         &lt;factor&gt;         &lt;variable&gt;         ident (pos)         &lt;selector&gt;         period (.)         ident (x)          rparent ())         semicolon (;)         &lt;statement&gt;         &lt;procedure/function-call&gt;         ident (writeln)         lparent (())         &lt;parameter-list&gt;         &lt;expression&gt;         &lt;simple-expression&gt;         &lt;term&gt;         &lt;factor&gt;         &lt;variable&gt;         ident (pos)         &lt;selector&gt;         period (.)         ident (y)          rparent ())         endsy (end)         period (.) </pre> |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Keterangan

Pengujian kesepuluh merupakan pengujian komprehensif yang menggabungkan hampir seluruh fitur utama yang telah diuji sebelumnya. Program ini memuat konstanta, type alias, array, enumerasi, record, prosedur, fungsi, nested loop, conditional statement, akses array, akses field record, serta pemanggilan fungsi dan prosedur. Output token menunjukkan bahwa lexer dapat mengenali berbagai jenis token dalam program panjang secara konsisten. Parse tree yang dihasilkan menunjukkan bahwa parser mampu membentuk struktur sintaks lengkap untuk

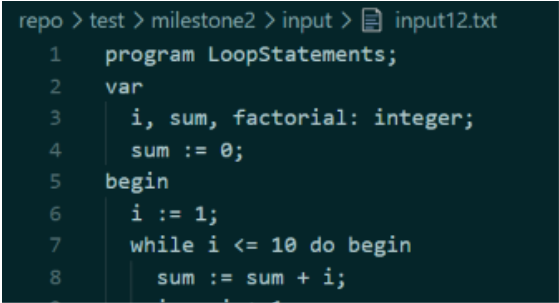
program berskala lebih besar dan membuktikan integrasi antar aturan grammar berjalan dengan baik.

### 3.11. Pengujian Kesebelas

| Input (input11.txt)                                                                                                                                                                                                                                                                                                                                        | Output                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>repo &gt; test &gt; milestone2 &gt; input &gt; input11.txt 1  program HelloWorld; 2  var 3    a, b: integer; 4  begin 5    a := 5; 6    b := a + 10 7    writeln('Result = ', b); 8  end.</pre>                                                                                                                                                       | <p>Lexing selesai.<br/>Output token disimpan di:<br/>test/milestone2/output/token11.txt<br/>[ParseError] Diharapkan 'ENDSY',<br/>mendapat 'IDENT' (writeln)</p> |
| Output (token11.txt)                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                 |
| <pre>repo &gt; test &gt; milestone2 &gt; output &gt; token11.txt You, 4 minutes ago   1 author (You) 1  programsy 2  ident (HelloWorld) 3  semicolon 4  varsy 5  ident (a) 6  comma 7  ident (b) 8  colon 9  ident (integer) 10 semicolon 11 beginsy 12 ident (a) 13 becomes 14 intcon (5) 15 semicolon 16 ident (b) 17 becomes 18 ident (a) 19 plus</pre> |                                                                                                                                                                 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| intcon (10)<br>ident (writeln)<br>lparent<br>string ('Result = ')<br>comma<br>ident (b)<br>rparent<br>semicolon<br>endsy<br>period                                                                                                                                                                                                                                                                                                                                                                                                             |  |
| <b>Keterangan</b><br><br>Pengujian kesebelas menunjukkan bagaimana parser melakukan penanganan <i>error</i> pada tingkat sintaks. <i>Test case</i> pada pengujian kesebelas ini dirancang sehingga baris kode Arion dengan operasi aritmatika dan operasi <i>assignment</i> tidak diakhiri oleh titik koma (;). Pesan <i>error</i> tersebut menyatakan bahwa apabila baris keenam yang tidak diakhiri dengan titik koma maka seharusnya token yang valid pada bagian tersebut adalah “end.”, sementara token yang terdeteksi adalah “ident b”. |  |

### 3.12. Pengujian Keduabelas

| Input (input12.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Output                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
|  <pre> repo &gt; test &gt; milestone2 &gt; input &gt; input12.txt 1  program LoopStatements; 2  var 3    i, sum, factorial: integer; 4    sum := 0; 5  begin 6    i := 1; 7    while i &lt;= 10 do begin 8      sum := sum + i; </pre> <pre> program LoopStatements; var   i, sum, factorial: integer;   sum := 0; begin   i := 1;   while i &lt;= 10 do begin     sum := sum + i;     i := i + 1   end;   factorial := 1;   i := 5;   repeat     factorial := factorial * i;     i := i - 1 </pre> | Lexing selesai.<br>Output token disimpan di:<br>test/milestone2/output/token12.txt<br>[ParseError] Diharapkan 'COLON',<br>mendapat 'BECOMES' (:=) |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre> until i == 0; for i := 10 downto 1 do   writeln(i); writeln(sum); writeln(factorial) end. </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |  |
| <p><b>Output (token12.txt)</b></p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |  |
| <div data-bbox="277 537 834 1031"> <pre> repo &gt; test &gt; milestone2 &gt; output &gt; token12.txt You, 8 minutes ago   1 author (You) 1  programsy 2  ident (LoopStatements) 3  semicolon 4  varsy 5  ident (i) 6  comma 7  ident (sum) 8  comma 9  ident (factorial) 10 colon 11 ident (integer) 12 semicolon 13 ident (sum) 14 becomes </pre> </div> <pre> programsy ident (LoopStatements) semicolon varsy ident (i) comma ident (sum) comma ident (factorial) colon ident (integer) semicolon ident (sum) becomes intcon (0) semicolon beginsy ident (i) becomes intcon (1) semicolon whilesy ident (i) leq intcon (10) </pre> |  |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <p>dosy<br/>beginsy<br/>ident (sum)<br/>becomes<br/>ident (sum)<br/>plus<br/>ident (i)<br/>semicolon<br/>ident (i)<br/>becomes<br/>ident (i)<br/>plus<br/>intcon (1)<br/>endsy<br/>semicolon<br/>ident (factorial)<br/>becomes<br/>intcon (1)<br/>semicolon<br/>ident (i)<br/>becomes<br/>intcon (5)<br/>semicolon<br/>repeatsy<br/>ident (factorial)<br/>becomes<br/>ident (factorial)<br/>times<br/>ident (i)<br/>semicolon<br/>ident (i)<br/>becomes<br/>ident (i)<br/>minus<br/>intcon (1)<br/>untilsy<br/>ident (i)<br/>eq<br/>intcon (0)<br/>semicolon<br/>forsy<br/>ident (i)<br/>becomes<br/>intcon (10)<br/>downtosy<br/>intcon (1)<br/>dosy<br/>ident (writeln)<br/>lparent<br/>ident (i)</p> |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| rp<br>parent<br>semicolon<br>ident (writeln)<br>lp<br>parent<br>ident (sum)<br>rp<br>parent<br>semicolon<br>ident (writeln)<br>lp<br>parent<br>ident (factorial)<br>rp<br>parent<br>endsy<br>period                                                                                                                                                                                                                                                                                                                                                                  |  |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |  |
| Pengujian keduabelas menunjukkan bagaimana parser melakukan penanganan <i>error</i> pada tingkat sintaks. <i>Test case</i> pada pengujian kesebelas ini dirancang sehingga baris kode operasi <i>assignment</i> dipindahkan ke bagian deklarasi. Pesan <i>error</i> tersebut menyatakan bahwa pada baris keempat pada <i>source code</i> , yang diharapkan adalah token titik dua, sementara token yang diberikan adalah token “BECOME” atau token “:=”. Kesalahan ini luput pada tingkat <i>lexer</i> , tetapi terdeteksi pada tingkat sintaks atau <i>parser</i> . |  |

### 3.13. Pengujian Ketigabelas

| Input (input13.txt)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Output                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> repo &gt; test &gt; milestone2 &gt; input &gt; input13.txt 1  program NestedAndComplex; 2  type 3      Grade == (A, B, C, D, E); 4      ScoreRange == 0..100; 5  var 6      score, i, j: integer; 7      grade: Grade; 8      matrix: array[1..3] of integer; 9  procedure PrintMatrix(size: integer); 10 var 11     k: integer; 12 begin </pre> <p> program NestedAndComplex;<br/> type<br/> Grade == (A, B, C, D, E);<br/> ScoreRange == 0..100;<br/> var<br/> score, i, j: integer;<br/> grade: Grade;<br/> matrix: array[1..3] of integer;<br/> procedure PrintMatrix(size: integer);<br/> var<br/> k: integer; </p> | Lexing selesai.<br>Output token disimpan di:<br>test/milestone2/output/token13.txt<br>[ParseError] Diharapkan 'ENDSY',<br>mendapat 'IDENT' (i) |

```

begin
  for k := 1 to size do
    writeln(matrix[k])
  end;
function GetGrade(s: integer): integer;
begin
  if s >= 80 then
    GetGrade := 1
  else if s >= 60 then
    GetGrade := 2
  else
    GetGrade := 3
  end;
begin
  for i := 1 to 3 do begin
    matrix[i] := i * 10;
  PrintMatrix(3);
  score := 75;
  case GetGrade(score) of
    1: writeln('A');
    2: writeln('B');
    3: writeln('C')
  end;
  i := 1;
  while i <= 3 do begin
    j := 1;
    while j <= 3 do begin
      writeln(i * j);
      j := j + 1
    end
    i := i + 1
  end
end.

```

**Output (token13.txt)**



repo > test > milestone2 > output > token13.txt

You, 8 minutes ago | 1 author (You)

```
1  programsy
2  ident (NestedAndComplex)
3  semicolon
4  typesy
5  ident (Grade)
6  eql
7  lparent
8  ident (A)
9  comma
10 ident (B)
11 comma
12 ident (C)
13 comma
14 ident (D)
15 comma
16 ident (E)
```

programsy  
ident (NestedAndComplex)  
semicolon  
typesy  
ident (Grade)  
eql  
lparent  
ident (A)  
comma  
ident (B)  
comma  
ident (C)  
comma  
ident (D)  
comma  
ident (E)  
rparent  
semicolon  
ident (ScoreRange)  
eql  
intcon (0)  
period  
period  
intcon (100)  
semicolon  
varsy  
ident (score)  
comma  
ident (i)  
comma  
ident (j)  
colon

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| ident (integer)<br>semicolon<br>ident (grade)<br>colon<br>ident (Grade)<br>semicolon<br>ident (matrix)<br>colon<br>arraysy<br>lbrack<br>intcon (1)<br>period<br>period<br>intcon (3)<br>rbrack<br>ofsy<br>ident (integer)<br>semicolon<br>proceduresy<br>ident (PrintMatrix)<br>lparent<br>ident (size)<br>colon<br>ident (integer)<br>rparent<br>semicolon<br>varsy<br>ident (k)<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>forsy<br>ident (k)<br>becomes<br>intcon (1)<br>tosy<br>ident (size)<br>dosy<br>ident (writeln)<br>lparent<br>ident (matrix)<br>lbrack<br>ident (k)<br>rbrack<br>rparent<br>endsy<br>semicolon<br>functionsy<br>ident (GetGrade) |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| lparent<br>ident (s)<br>colon<br>ident (integer)<br>rparent<br>colon<br>ident (integer)<br>semicolon<br>beginsy<br>ifsy<br>ident (s)<br>geq<br>intcon (80)<br>thensy<br>ident (GetGrade)<br>becomes<br>intcon (1)<br>elsesy<br>ifsy<br>ident (s)<br>geq<br>intcon (60)<br>thensy<br>ident (GetGrade)<br>becomes<br>intcon (2)<br>elsesy<br>ident (GetGrade)<br>becomes<br>intcon (3)<br>endsy<br>semicolon<br>beginsy<br>forsy<br>ident (i)<br>becomes<br>intcon (1)<br>tosy<br>intcon (3)<br>dosy<br>beginsy<br>ident (matrix)<br>lbrack<br>ident (i)<br>rbrack<br>becomes<br>ident (i)<br>times<br>intcon (10)<br>semicolon |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| ident (PrintMatrix)<br>lparent<br>intcon (3)<br>rparent<br>semicolon<br>ident (score)<br>becomes<br>intcon (75)<br>semicolon<br>casesy<br>ident (GetGrade)<br>lparent<br>ident (score)<br>rparent<br>ofsy<br>intcon (1)<br>colon<br>ident (writeln)<br>lparent<br>charcon ('A')<br>rparent<br>semicolon<br>intcon (2)<br>colon<br>ident (writeln)<br>lparent<br>charcon ('B')<br>rparent<br>semicolon<br>intcon (3)<br>colon<br>ident (writeln)<br>lparent<br>charcon ('C')<br>rparent<br>endsy<br>semicolon<br>ident (i)<br>becomes<br>intcon (1)<br>semicolon<br>whilesy<br>ident (i)<br>leq<br>intcon (3)<br>dosy<br>beginsy<br>ident (j)<br>becomes<br>intcon (1) |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| semicolon<br>whilesy<br>ident (j)<br>leq<br>intcon (3)<br>dosy<br>beginsy<br>ident (writeln)<br>lparent<br>ident (i)<br>times<br>ident (j)<br>rparent<br>semicolon<br>ident (j)<br>becomes<br>ident (j)<br>plus<br>intcon (1)<br>ident (i)<br>becomes<br>ident (i)<br>plus<br>intcon (1)<br>endsy<br>endsy<br>period                                                                                                                                                                                                                                                     |  |
| <b>Keterangan</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |  |
| Pengujian ketigabelas menunjukkan bagaimana parser melakukan penanganan <i>error</i> pada tingkat sintaks. <i>Test case</i> pada pengujian ketigabelas ini dirancang sehingga salah satu token “ENDSY” atau “end.” pada baris ke-41 dihilangkan dari blok “while”. Pesan error tersebut menyatakan bahwa pada baris ke-41 tersebut token yang diharapkan sesuai aturan produksi adalah “ENDSY” atau “end.”, tetapi token yang diberikan oleh <i>source code</i> melalui <i>lexer</i> adalah “IDENT i” sebagaimana baris ke-41 adalah potongan kode berikut “i := i + 1”. |  |

## BAB IV KESIMPULAN DAN SARAN

### 4.1. Kesimpulan

Pada *milestone* ini telah berhasil dirancang dan diimplementasikan *parser* untuk bahasa Arion menggunakan metode Recursive Descent Parsing. *Parser* dibangun di atas hasil *lexical analyzer* yang telah dikembangkan sebelumnya sehingga keseluruhan proses analisis *source code* dapat dilakukan mulai dari tokenisasi hingga pembentukan *parse tree*. Implementasi *parser* dilakukan menggunakan bahasa C++ dengan pendekatan *top-down parsing* yang memanfaatkan fungsi-fungsi rekursif sesuai aturan produksi grammar.

Perancangan *parse tree* berhasil menyediakan representasi struktur sintaks program secara hierarkis melalui node *terminal*, *non-terminal*, dan *error*. Penggunaan struktur data berbasis *tree* mempermudah proses visualisasi hasil *parsing* serta membantu proses *debugging* ketika terjadi kesalahan sintaks. Selain itu, penggunaan `std::unique_ptr` pada *child node* membuat pengelolaan memori menjadi lebih aman dan terstruktur.

*Parser* berhasil mengimplementasikan aturan produksi Context-Free Grammar (CFG) bahasa Arion dan mampu mengenali berbagai konstruksi bahasa, seperti deklarasi konstanta, tipe, variabel, *array*, *record*, subprogram, percabangan, perulangan, *procedure/function call*, serta ekspresi aritmatika dan logika dengan prioritas operator yang benar. Mekanisme *exception handling* juga memungkinkan *parser* memberikan pesan *error* yang informatif beserta lokasi kesalahan pada *source code*.

Berdasarkan hasil pengujian, sistem mampu memproses berbagai kasus program sederhana hingga kompleks secara konsisten. Seluruh pengujian menunjukkan bahwa *lexer* dan *parser* dapat terintegrasi dengan baik dalam membangun *parse tree* sesuai *grammar* yang telah ditentukan. Dengan demikian, *milestone 2* ini berhasil memenuhi tujuan utama, yaitu melakukan analisis sintaks terhadap *source code* Arion dan menghasilkan representasi *parse tree* yang valid sebagai dasar pengembangan tahap *compiler* berikutnya.

### 4.2. Saran

`ParseException` saat ini hanya menyertakan pesan, tetapi kelasnya sudah mendukung `line` dan `column`. Akan lebih baik jika `expect()` dan `consume()` meneruskan posisi token ke `exception`, sehingga pesan *error* bisa lebih informatif. Selain itu, di `parse_tree.cpp`, `makeTerminal(const Token&)` mengubah label token menjadi lowercase sehingga perlu diperhatikan bahwa label *tree* tidak selalu sama dengan token `type` asli jika dibutuhkan *debugging* token secara eksplisit.

## LAMPIRAN

Link GitHub: <https://github.com/alyanrrhma/MAR-Tubes-IF2224-2026>

Tabel Pembagian Tugas:

| No. | NIM - Nama                       | Tugas                                                                 | Persentase |
|-----|----------------------------------|-----------------------------------------------------------------------|------------|
| 1.  | 13524017 - Aziza Dharma Putri    | 1. Implementasi parser<br>2. Laporan bagian pengujian                 | 25%        |
| 2.  | 13524053 Ahmad Zaky Robbani      | 1. Revisi perbaikan lexer<br>2. Laporan bagian kesimpulan dan saran   | 25%        |
| 3.  | 13524063 - Marcel Luther Sitorus | 1. Laporan bagian teori dan rancangan<br>2. Main.cpp dan Makefile     | 25%        |
| 4.  | 13524081 - Alya Nur Rahmah       | 1. Implementasi struktur parse tree<br>2. Laporan bagian implementasi | 25%        |

## DAFTAR PUSTAKA

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson Education.
- TutorialsPoint. (n.d.). *Compiler Design - Lexical Analysis*. Accessed online from [https://www.tutorialspoint.com/compiler\\_design/compiler\\_design\\_lexical\\_analysis.htm](https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm)
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson Education. Accessed online from [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)
- Free Software Foundation. (n.d.). *An Introduction to Makefiles*. Accessed online from [https://www.gnu.org/software/make/manual/html\\_node/Introduction.html](https://www.gnu.org/software/make/manual/html_node/Introduction.html)
- Theicfire. (n.d.). *Makefile Tutorial by Example*. Accessed online from <https://makefiletutorial.com/>
- Judhi S. (n.d.). *Parse Trees and Conversions*. IF 2124 Teori Bahasa Formal Otomata. Accessed online from [https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362559-Module-06/file/1757509600732\\_slide-08?sv=2024-11-04&spr=https&st=2025-09-10T13%3A06%3A40Z&se=2027-09-10T13%3A06%3A40Z&sr=b&sp=r&sig=BfWyl7SqCHFm9Nistwd7qRj0k6k44O53cYS9fUwXqLQ%3D&rscet=application%2Fpdf](https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362559-Module-06/file/1757509600732_slide-08?sv=2024-11-04&spr=https&st=2025-09-10T13%3A06%3A40Z&se=2027-09-10T13%3A06%3A40Z&sr=b&sp=r&sig=BfWyl7SqCHFm9Nistwd7qRj0k6k44O53cYS9fUwXqLQ%3D&rscet=application%2Fpdf)